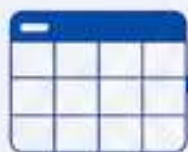


Fácil Assim



MySQL™

```
-- Seleciona todos os registros  
SELECT *  
FROM clientes  
WHERE ativo = 1  
ORDER BY nome;
```



CLIENTE
id_cliente (PK)
nome
email
telefone

PEDIDO
id_pedido (PK)
id_cliente (FK)
data_pedido
valor_total

ITEM_PEDIDO
id_item (PK)
id_pedido (FK)
id_produto (FK)
quantidade
valor_unitario

BANCO DE DADOS

LINGUAGEM SQL

DDL • DML • DQL • TABELAS • CONSULTAS • RELACIONAMENTOS



MATERIAL
DIDÁTICO



Prof. **Enildo**

2026

Sumário

CAPÍTULO 1 – INTRODUÇÃO À LINGUAGEM SQL	4
Introdução	4
Linguagem SQL	4
O que é um Banco de Dados Relacional?.....	5
Sistema Gerenciador de Banco de Dados	5
Ambiente MySQL	6
SQL é igual em todos os Bancos de Dados?	7
Subconjuntos da Linguagem SQL.....	7
RESUMO	8
CAPÍTULO 2 – IMPLEMENTAÇÃO DO BANCO DE DADOS NO MYSQL.....	8
Comentário em uma linha	8
Comentário em várias linhas	9
Como ler as sintaxes apresentadas na apostila	9
Criação e Exclusão de Banco de Dados	9
SHOW DATABASES – Visualizando os Bancos Existentes	10
USE – Selecionando um Banco de Dados	10
DROP DATABASE – Excluindo um Banco de Dados	11
RESUMO	12
CAPÍTULO 3 – DDL – CRIAÇÃO E ESTRUTURA DAS TABELAS	12
Tipos de Dados	12
Tipos de Dados no MySQL	13
DDL – Data Definition Language.....	18
CREATE TABLE – Criando uma Tabela	18
SHOW TABLES – Visualizando as Tabelas.....	20
DESCRIBE – Visualizando a Estrutura da Tabela	21
ALTER TABLE – Alterando uma Tabela	23
MODIFY – Modificando uma Coluna.....	24
DROP COLUMN – Excluindo uma Coluna.....	25
RENAME COLUMN – Renomeando uma Coluna	26
SQL Constraints – Restrições	27
Chave Primária Composta	32
FOREIGN KEY – Chave Estrangeira	35
DROP TABLE – Excluindo uma Tabela	38
TRUNCATE TABLE	40
RENAME TABLE – Renomeando uma Tabela	41
RESUMO	42
DO PROJETO AO BANCO DE DADOS FÍSICO.....	42
EXERCÍCIOS PRÁTICOS	43

ETAPA 1 — FIXAÇÃO DOS PRIMEIROS COMANDOS	43
Exercício Prático 01 – Preparando os Ambientes	43
Exercício Prático 02 – Um Novo Projeto	44
Exercício Prático 03 – Lendo o Projeto da Oficina	44
Exercício Prático 04 – Projeto x Implementação	45
Exercício Prático 05 – Mudança nas Regras de Negócio.....	45
Exercício Prático 06 – Projeto Cancelado	45
Exercício Prático 07 – Ambientes de Homologação.....	46
Exercício Prático 08 – Primeira Auditoria	46
ETAPA 2 — PROJETO, RELACIONAMENTOS E IMPLEMENTAÇÃO	47
Exercício Prático 09 – Projeto da Escola de Idiomas	47
Exercício Prático 10 – Do Relacionamento 1:N para o Banco Físico	47
Exercício Prático 11 – Professor Responsável.....	48
Exercício Prático 12 – Cadastro de Alunos	48
Exercício Prático 13 – Do N:N para uma Tabela Associativa	48
Exercício Prático 14 – Por que Normalizamos?	49
Exercício Prático 15 – O Projeto Está Certo, mas o SQL Está?	49
ETAPA 3 — DESAFIO DE INTEGRAÇÃO	50
Exercício Prático 16 – Projeto de uma Pequena Locadora	50
Exercício Prático 17 – Mudanças na Locadora	51
Exercício Prático 18 – Estrutura Temporária.....	51
Exercício Prático 19 – Encerramento dos Ambientes de Treinamento.....	52
ETAPA 4 — PROJETO INTEGRADOR.....	52
Exercício Prático 20 – Retomando o Projeto Antes do SQL.....	52
Exercício Prático 21 – Do Modelo Conceitual para o Lógico	53
Exercício Prático 22 – Nascimento do Banco Oficial.....	53
Exercício Prático 23 – Implementando Localização	54
Exercício Prático 24 – Implementando Clientes e Endereços	54
Exercício Prático 25 – Estrutura Organizacional	55
Exercício Prático 26 – Catálogo Comercial	55
Exercício Prático 27 – Fornecedores e Produtos	56
Exercício Prático 28 – Controle de Estoque	56
Exercício Prático 29 – Pedidos	57
Exercício Prático 30 – Itens do Pedido.....	57
Exercício Prático 31 – Pagamentos	57
Exercício Prático 32 – Entregas	58
Exercício Prático 33 – Auditoria Projeto x Banco Físico.....	58
Exercício Prático 34 – O Negócio Mudou.....	58
Exercício Prático 35 – Mudanças em Produtos.....	59

Exercício Prático 36 – Outra Mudança de Regra	59
Exercício Prático 37 – Informação Descontinuada.....	59
Exercício Prático 38 – Estrutura Temporária de Importação	59
Exercício Prático 39 – Alteração de Terminologia.....	60
Exercício Prático 40 – Desafio Final de DDL.....	60
INDEX – Índices.....	62

CAPÍTULO 1 – INTRODUÇÃO À LINGUAGEM SQL

Introdução

Este material foi produzido com a finalidade de auxiliar nas aulas de **Banco de Dados – Linguagem SQL**, oferecendo uma abordagem prática e acessível para quem está iniciando seus estudos na área.

Ao longo dos próximos capítulos serão apresentados comandos da linguagem **SQL padrão ANSI**, utilizando como base o Sistema Gerenciador de Banco de Dados (SGBD) **MySQL**, operado preferencialmente pela interface gráfica **MySQL Workbench**.

Por utilizarmos o MySQL, aparecerão alguns comandos e recursos específicos deste SGBD. Mas não se preocupe! Quando um comando não fizer parte do padrão SQL apresentado no material, isso será indicado. Caso esteja utilizando outro SGBD, será necessário verificar a sintaxe equivalente.

Nosso objetivo será aprender SQL de forma progressiva, começando pelos conceitos fundamentais e avançando para a criação das estruturas do banco de dados, manipulação das informações e realização de consultas.

Linguagem SQL

Existem inúmeras linguagens no mercado de Tecnologia da Informação: linguagens de programação orientadas a objetos, estruturadas, linguagens de marcação de texto, entre outras.

Quando trabalhamos com **bancos de dados relacionais**, uma das principais linguagens utilizadas é a SQL.

SQL é a sigla para:

- **Structured Query Language**

Em português:

- **Linguagem de Consulta Estruturada**

A SQL é uma linguagem utilizada para trabalhar com bancos de dados relacionais. Por meio dela podemos criar estruturas, armazenar informações, alterar dados, excluir registros e realizar consultas.

Sua aplicação é bastante ampla no mercado de banco de dados e desenvolvimento de sistemas, sendo importante para programadores, técnicos, analistas e administradores de banco de dados.

A linguagem SQL segue padrões estabelecidos por organizações como ANSI (American National Standards Institute) e ISO (International Organization for Standardization), permitindo que seus principais conceitos e comandos mantenham uma base comum entre diferentes Sistemas Gerenciadores de Banco de Dados.



O que é um Banco de Dados Relacional?

Um **banco de dados relacional** é um banco no qual as informações são organizadas em estruturas que podem se relacionar entre si.

Normalmente, essas informações são armazenadas em **tabelas**.

Uma tabela é organizada em linhas e colunas. Cada coluna representa uma característica da informação armazenada e cada linha representa um registro.

Por exemplo, imagine uma tabela utilizada para armazenar clientes:

ClienteID	Nome	Telefone
1	Carlos	(43) 9999-1111
2	Maria	(43) 9999-2222
3	João	(43) 9999-3333

Nesse exemplo:

Cliente → é a tabela utilizada para armazenar os dados.

ClienteID, Nome e Telefone → são campos ou colunas da tabela.

Cada linha → representa um registro armazenado.

O grande diferencial de um banco de dados relacional é que uma tabela pode possuir relação com outra tabela.

Por exemplo, podemos possuir uma tabela para armazenar os clientes e outra para armazenar os pedidos realizados por eles.



Observe que `ClienteID` aparece nas duas tabelas.

Não se preocupe com as siglas PK e FK neste momento. Mais adiante aprenderemos como as chaves primárias e estrangeiras são utilizadas para estabelecer relacionamentos entre tabelas.

Sistema Gerenciador de Banco de Dados

É o software responsável por gerenciar os bancos de dados.

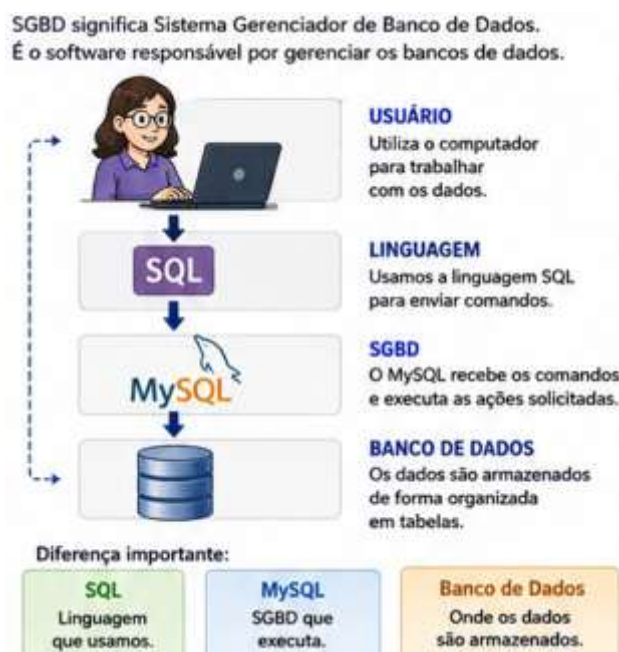
O SGBD permite armazenar, organizar, recuperar e manter as informações existentes no banco.

Existem diversos SGBDs disponíveis no mercado, como MySQL, Oracle Database, Microsoft SQL Server e PostgreSQL.

Em nossas aulas utilizaremos o MySQL.

Portanto, é importante entender a diferença:

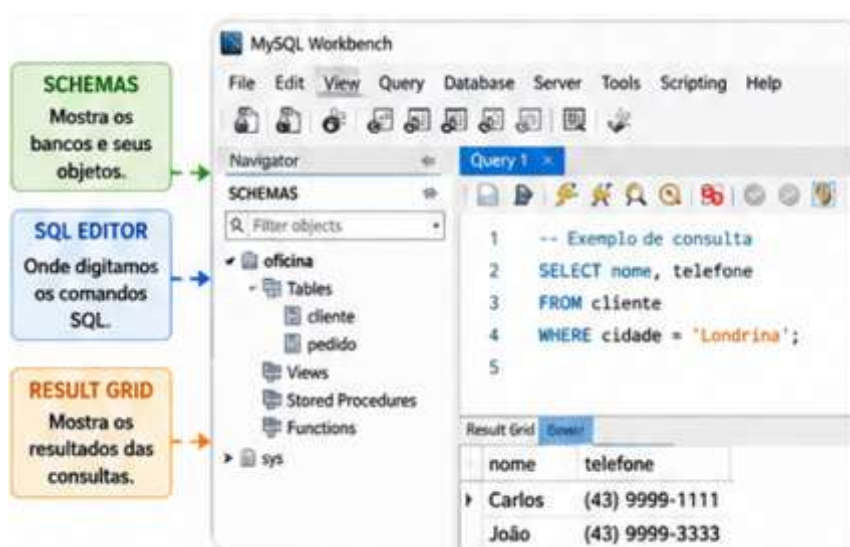
Termo	O que representa
Banco de Dados	Conjunto organizado de dados
SGBD	Sistema que gerencia os bancos de dados
SQL	Linguagem utilizada para trabalhar com os dados e suas estruturas
MySQL	SGBD utilizado em nossas aulas



Ambiente MySQL

Para executar os comandos SQL utilizaremos o **MySQL**.
Existem diferentes ferramentas que permitem trabalhar com esse SGBD.

Em nossas aulas utilizaremos preferencialmente o MySQL Workbench.
O MySQL Workbench possui uma interface gráfica que facilita a criação e administração dos bancos de dados e a execução dos comandos SQL.



SQL é igual em todos os Bancos de Dados?

Inicialmente podemos imaginar que a linguagem SQL seja exatamente igual para todos os bancos de dados. Porém, isso não ocorre em todos os casos.

Existe uma base padronizada da linguagem SQL, mas os fabricantes dos Sistemas Gerenciadores de Banco de Dados podem implementar recursos e extensões próprias.

Geralmente utilizamos a expressão SQL padrão ANSI/ISO para nos referirmos à base padronizada da linguagem.

Alguns fabricantes também possuem extensões próprias. Podemos citar como exemplos:

- **Oracle** → PL/SQL
- **Microsoft SQL Server** → Transact-SQL (T-SQL)



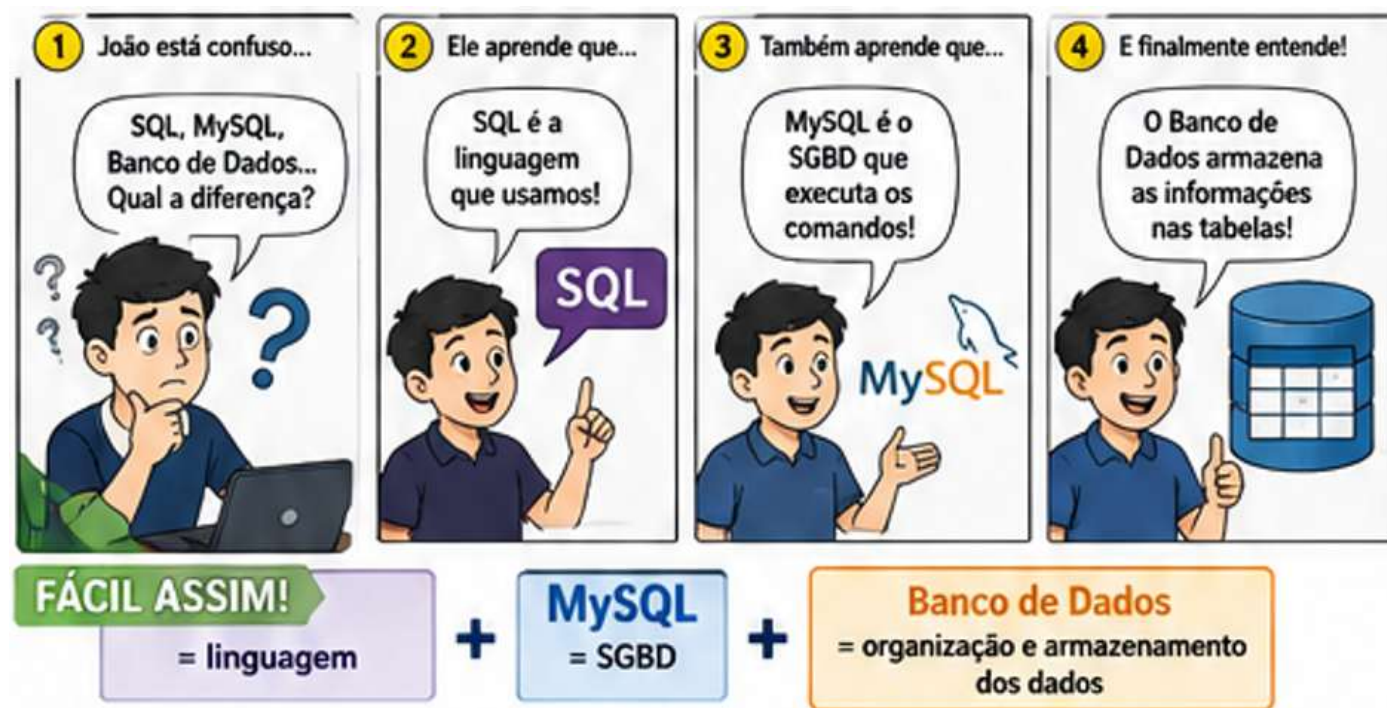
Isso significa que, ao aprender SQL, grande parte do conhecimento poderá ser utilizada em diferentes bancos de dados. Porém, alguns comandos, funções e recursos podem possuir diferenças. Em nosso material, o SGBD de referência será o **MySQL 8.x**.

Subconjuntos da Linguagem SQL

Os comandos da linguagem SQL podem ser divididos em grupos de acordo com aquilo que realizam.



RESUMO



CAPÍTULO 2 – IMPLEMENTAÇÃO DO BANCO DE DADOS NO MYSQL

No capítulo anterior conhecemos os conceitos básicos relacionados à linguagem SQL, aos bancos de dados relacionais e ao MySQL.

A partir deste capítulo começaremos a trabalhar diretamente com comandos SQL no **MySQL Workbench**. Antes de criarmos nosso primeiro banco de dados, vamos conhecer alguns recursos importantes para escrever e compreender os comandos apresentados durante as aulas.

Comentários na Query

Antes de começarmos, vamos aprender como fazer comentários em um arquivo do tipo `.sql`.

Um arquivo `.sql` é um arquivo de texto que pode armazenar comandos SQL. Durante as aulas, podemos utilizar comentários para inserir explicações, observações e lembretes junto aos comandos.

Os comentários não são executados pelo MySQL. Comentário em uma linha

Para escrever um comentário em uma única linha, podemos utilizar dois hífens seguidos de espaço:

```
-- Este é um comentário em uma linha
```

Por exemplo:

```
-- Criando o banco de dados da escola  
CREATE DATABASE dbEscola;
```

A primeira linha é apenas um comentário.

A segunda linha contém um comando SQL e será executada.

Comentário em várias linhas

Quando o comentário ocupar mais de uma linha, podemos utilizar `/*` para iniciar e `*/` para finalizar:

```
/*  
Este é um comentário  
que ocupa várias linhas.  
*/
```

Como ler as sintaxes apresentadas na apostila

Durante o aprendizado de SQL encontraremos estruturas chamadas de **sintaxes**.
A sintaxe mostra como um comando deve ser escrito.

Por exemplo:

```
CREATE DATABASE <nome_base> [;]
```

Alguns símbolos serão utilizados para facilitar a leitura dessas sintaxes.

Símbolo	Significado
< >	Indica uma informação que deverá ser substituída
[]	Indica um elemento opcional na representação da sintaxe
	Indica alternativas entre opções

Exemplo

Observe:

```
CREATE DATABASE <nome_base> [;]
```

`<nome_base>` indica que devemos substituir esse trecho pelo nome que desejamos dar ao banco.

Por exemplo:

```
CREATE DATABASE dbEscola;
```

Criação e Exclusão de Banco de Dados

Agora podemos começar efetivamente a trabalhar com os primeiros comandos SQL.

Nosso primeiro objetivo será:

criar um banco → verificar sua existência → selecionar o banco → excluir o banco quando necessário.

CREATE DATABASE – Criando um Banco de Dados

O comando `CREATE DATABASE` é utilizado para criar um banco de dados.

Sintaxe

```
CREATE DATABASE <nome_base>;
```

Vamos entender cada parte:

CREATE → comando utilizado para criar objetos.

DATABASE → indica que o objeto que desejamos criar é um banco de dados.

<nome_base> → local onde informamos o nome que desejamos dar ao banco.

; → marca o final do comando no script e permite separar uma instrução da próxima.

Exemplo

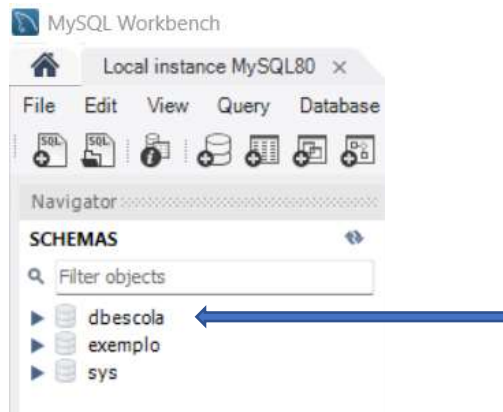
Vamos criar um banco chamado `dbEscola`:

```
CREATE DATABASE dbEscola;
```

Ao executar esse comando, o MySQL criará o banco de dados.

Resultado esperado

No MySQL Workbench, o banco poderá ser visualizado na área **SCHEMAS**.



CREATE DATABASE → cria o banco → aparece em **SCHEMAS**

SHOW DATABASES – Visualizando os Bancos Existentes

Depois de criar um banco, podemos verificar os bancos existentes no servidor MySQL.

Utilizamos:

```
SHOW DATABASES;
```

SHOW DATABASES lista todos os bancos disponíveis no servidor.

Exemplo de resultado

Database
dbescola
exemplo
information_schema
mysql
performance_schema
sys

Atenção: `SHOW DATABASES` é uma instrução específica do MySQL para listar os bancos de dados disponíveis no servidor. Ela não faz parte do conjunto padronizado de comandos SQL ANSI/ISO

USE – Selecionando um Banco de Dados

Criar um banco não significa que ele automaticamente será o banco utilizado pelos próximos comandos. Para selecionar o banco com o qual desejamos trabalhar, podemos utilizar:

```
USE <nome_base>;
```

Exemplo

```
USE dbEscola;
```

Isso informa ao MySQL que os próximos comandos deverão utilizar `dbEscola` como banco de dados padrão da sessão.

DROP DATABASE – Excluindo um Banco de Dados

Assim como podemos criar um banco de dados, também podemos excluí-lo. Para isso utilizamos o comando `DROP DATABASE`.

Sintaxe

```
DROP DATABASE <nome_base>;
```

Vamos entender:

DROP → comando utilizado para excluir um objeto.

DATABASE → indica que o objeto que será excluído é um banco de dados.

<nome_base> → nome do banco que desejamos excluir.

Exemplo

```
DROP DATABASE dbEscola;
```

Após a execução, o banco `dbEscola` será excluído.

⚠ ATENÇÃO

`DROP DATABASE` exclui o banco de dados e seus objetos. Portanto, esse comando deve ser utilizado com cuidado.

DROP DATABASE IF EXISTS

Se tentarmos excluir um banco que não existe, o MySQL retornará um erro.

Podemos verificar sua existência antes da exclusão utilizando:

```
DROP DATABASE IF EXISTS dbEscola;
```

`IF EXISTS` significa:

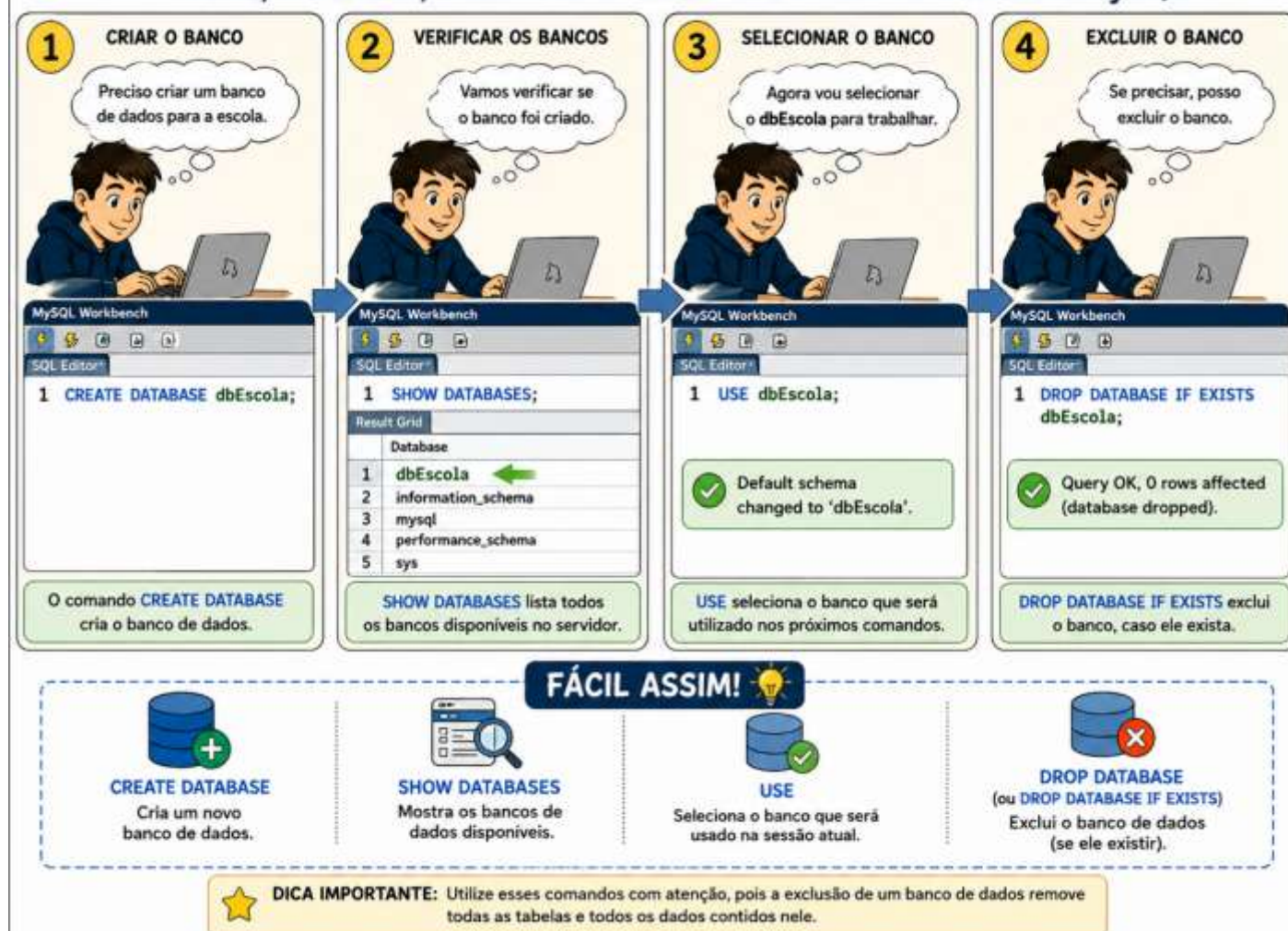
“se existir”.

Portanto, podemos ler o comando como:

“Exclua o banco `dbEscola` se ele existir.”

RESUMO

Criar, visualizar, selecionar e excluir um banco de dados no MySQL



CAPÍTULO 3 – DDL – CRIAÇÃO E ESTRUTURA DAS TABELAS

Tipos de Dados

Antes de criarmos nossas primeiras tabelas, precisamos entender uma característica muito importante. Ao criar uma tabela, devemos informar **qual tipo de informação poderá ser armazenada em cada coluna**. Imagine uma tabela de alunos:

AlunoID	Nome	DataNascimento	Mensalidade
1	Carlos	2005-08-15	450.00
2	Maria	2004-11-20	520.00

Observe que cada coluna armazena um tipo diferente de informação:

- **AlunoID** → número inteiro
- **Nome** → texto
- **DataNascimento** → data
- **Mensalidade** → número com casas decimais

É justamente para isso que utilizamos os **tipos de dados**.

Tipo de dado é a definição que informa ao banco de dados qual tipo de valor poderá ser armazenado em uma coluna.

Por exemplo:

AlunoID INT

Nome VARCHAR (60)

DataNascimento DATE

Mensalidade DECIMAL (10, 2)

Não se preocupe ainda com a criação da tabela. Primeiro vamos conhecer os principais tipos que utilizaremos.

Tipos de Dados no MySQL

O MySQL possui diversos tipos de dados. Para facilitar nosso aprendizado, inicialmente vamos dividi-los em três grupos principais:

Numéricos → **Texto** → **Data e Hora**



Tipos Numéricos Inteiros

Os tipos numéricos inteiros são utilizados para armazenar números **sem casas decimais**.

Exemplos:

1

25

150

2026

O MySQL possui diferentes tipos para números inteiros:

Tipo	Espaço	Exemplo de utilização
TINYINT	1 byte	pequenas quantidades
SMALLINT	2 bytes	números inteiros pequenos
MEDIUMINT	3 bytes	números inteiros médios
INT OU INTEGER	4 bytes	números inteiros em geral
BIGINT	8 bytes	números inteiros muito grandes

Esses tamanhos e categorias estão de acordo com a documentação atual do MySQL.

Para nossas primeiras tabelas, o tipo que utilizaremos com maior frequência será:
INT

Por exemplo:

`AlunoID INT`

Isso indica que a coluna `AlunoID` armazenará números inteiros.

Fácil Assim!

INT = número inteiro

1

10

150

2026

Tipos Numéricos com Casas Decimais

Nem todos os números são inteiros.

Imagine que precisamos armazenar o preço de um produto:

49.90

ou o valor de uma mensalidade:

525.75

Nesse caso, podemos utilizar:

`DECIMAL (M, D)`

Onde:

M → quantidade máxima de dígitos do número.

D → quantidade de dígitos depois do ponto decimal.

Exemplo

`DECIMAL (10, 2)`

Isso permite armazenar números com até **10 dígitos no total**, sendo **2 depois do ponto decimal**. Essa definição coincide com a sintaxe atual do MySQL.

Exemplo:

1250.50

Para valores que precisam de precisão, como **preços, salários e valores monetários**, `DECIMAL` será nossa escolha preferencial.

O MySQL também possui os tipos:

`FLOAT`

`DOUBLE`

Eles trabalham com valores numéricos aproximados e serão utilizados quando houver necessidade desse tipo de representação. `FLOAT` e `DOUBLE` não devem ser apresentados ao aluno como equivalentes a `DECIMAL`: os dois primeiros representam valores aproximados, enquanto `DECIMAL` trabalha com valores exatos.

Fácil Assim!

`INT` → números inteiros

`DECIMAL` → números com casas decimais quando precisamos de precisão

Tipos de Dados para Texto

Para armazenar textos, utilizaremos principalmente:

CHAR

VARCHAR

CHAR

O tipo CHAR é utilizado para textos de **tamanho fixo**.

Sintaxe:

CHAR (tamanho)

Exemplo:

Estado CHAR (2)

Podemos armazenar:

PR

SP

SC

RJ

Como uma sigla de estado possui sempre dois caracteres, CHAR (2) é uma escolha adequada.

VARCHAR

O tipo VARCHAR é utilizado para textos de **tamanho variável**.

Sintaxe:

VARCHAR (tamanho)

Exemplo:

Nome VARCHAR (60)

Nesse caso, a coluna poderá armazenar nomes com comprimentos diferentes:

Ana

Carlos

Maria Fernanda

O número informado em VARCHAR (60) representa o **comprimento máximo em caracteres** definido para aquela coluna. No MySQL, o limite efetivo também depende do tamanho máximo da linha e do conjunto de caracteres utilizado.

Comparando

Tipo	Característica	Exemplo
CHAR (2)	tamanho fixo	PR
VARCHAR (60)	tamanho variável	Carlos da Silva

Fácil Assim!

CHAR → tamanho fixo

VARCHAR → tamanho variável

Tipos de Data e Hora

O MySQL também possui tipos específicos para armazenar datas e horários.

Os principais que conheceremos inicialmente são:

Tipo	Armazena	Exemplo
DATE	data	2026-08-28
TIME	hora ou intervalo de tempo	18:30:00
DATETIME	data e hora	2026-08-28 18:30:00
YEAR	ano	2026

DATE

Utilizado quando precisamos armazenar uma **data**.

DataNascimento DATE

Exemplo de valor:

2005-08-15

No MySQL, o formato normalmente exibido para DATE é:

AAAA-MM-DD

TIME

Utilizado para valores de tempo.

Horario TIME

Exemplo:

19:30:00

DATETIME

Armazena **data e hora** juntas.

DataCadastro DATETIME

Exemplo:

2026-08-28 18:30:00

DATE e DATETIME possuem comportamentos e faixas próprios documentados pelo MySQL.

YEAR

Utilizado para armazenar um ano.

AnoConclusao YEAR

Exemplo:

2026

No MySQL atual, devemos utilizar simplesmente YEAR; não há necessidade de ensinar YEAR(4).

E o tipo BOOLEAN?

Também encontraremos situações em que precisamos representar uma condição como:

Ativo / Inativo

Sim / Não

Verdadeiro / Falso

No MySQL podemos escrever:

BOOLEAN

Por exemplo:

Ativo BOOLEAN

Entretanto, existe uma particularidade importante do MySQL:

BOOLEAN e BOOL são tratados como equivalentes ao tipo TINYINT.

Neste momento, basta guardar a ideia:

BOOLEAN → utilizado para representar uma condição lógica.

Mais adiante veremos sua utilização em uma tabela.

Escolhendo o Tipo de Dado

Antes de criar uma coluna, devemos pensar:

“Que informação será armazenada aqui?”

Por exemplo:

Informação	Tipo que podemos utilizar
Código do aluno	INT
Nome	VARCHAR (60)
Estado	CHAR (2)
Data de nascimento	DATE
Mensalidade	DECIMAL (10,2)
Data e hora do cadastro	DATETIME
Ativo	BOOLEAN

RESUMO



Antes de criar uma coluna, pense no dado que ela deverá armazenar. O tipo de dado define quais valores aquela coluna foi criada para receber.

DDL – Data Definition Language

DDL é a sigla para:
Data Definition Language

Em português:
Linguagem de Definição de Dados

A DDL reúne comandos utilizados para trabalhar com as **estruturas do banco de dados**, permitindo criar, alterar ou excluir objetos.

Entre os principais comandos que estudaremos estão:

- **CREATE** → cria objetos
- **ALTER** → altera objetos existentes
- **DROP** → exclui objetos

Já utilizamos dois deles no capítulo anterior:

```
CREATE DATABASE dbEscola;
```

```
DROP DATABASE dbEscola;
```

Agora utilizaremos a DDL para trabalhar com outro objeto muito importante do banco de dados:
as tabelas.

CREATE TABLE – Criando uma Tabela

O comando `CREATE TABLE` é utilizado para criar uma nova tabela.

Antes de executar esse comando, devemos estar utilizando o banco de dados correto.

Por exemplo:

```
USE dbEscola;
```

Agora podemos criar nossas tabelas dentro desse banco.

Sintaxe

```
CREATE TABLE <nome_tabela> (  
    <nome_campo> <tipo>,  
    <nome_campo> <tipo>,  
    ...  
);
```

Vamos entender cada parte:

- **CREATE** → comando utilizado para criar objetos.
- **TABLE** → indica que o objeto que desejamos criar é uma tabela.
- **<nome_tabela>** → local onde informamos o nome da tabela.
- **()** → entre os parênteses informamos as colunas que farão parte da tabela.
- **<nome_campo>** → nome da coluna.
- **<tipo>** → tipo de dado que será armazenado naquela coluna.
- **,** → separa a definição de uma coluna da próxima.
- **;** → marca o final do comando no script.

Declarando uma coluna

Vamos observar apenas esta parte:

<nome_campo> <tipo>

Por exemplo:

AlunoID INT

Podemos ler assim:

- **AlunoID** → nome da coluna.
- **INT** → tipo de dado que poderá ser armazenado.

Outro exemplo:

Nome VARCHAR (60)

- **Nome** → nome da coluna.
- **VARCHAR(60)** → permite armazenar um texto com tamanho variável de até 60 caracteres.

Agora podemos juntar várias colunas para formar uma tabela.

Exemplo – Criando a Tabela Aluno

Vamos criar uma tabela chamada `tbAluno`.

Ela terá inicialmente quatro colunas:

Coluna	Tipo	Informação armazenada
AlunoID	INT	código do aluno
Nome	VARCHAR (60)	nome do aluno
DataNascimento	DATE	data de nascimento
Estado	CHAR (2)	sigla do estado

Comando:

```
CREATE TABLE tbAluno (  
    AlunoID INT,  
    Nome VARCHAR(60),  
    DataNascimento DATE,  
    Estado CHAR(2)  
);
```

Observe com atenção as vírgulas.

```
AlunoID INT,  
Nome VARCHAR (60) ,  
DataNascimento DATE,  
Estado CHAR (2)
```

Utilizamos a vírgula para separar as colunas.

Na **última coluna**, não colocamos vírgula antes de fechar o parêntese.

Podemos imaginar o comando desta maneira:

```
CREATE TABLE tbAluno
```

↓

criar tabela

```
(
```

```
  AlunoID INT,  
  Nome VARCHAR(60),  
  DataNascimento DATE,  
  Estado CHAR(2)
```

```
);
```

↓

suas colunas

Depois da execução, a tabela `tbAluno` passa a existir dentro do banco `dbEscola`.

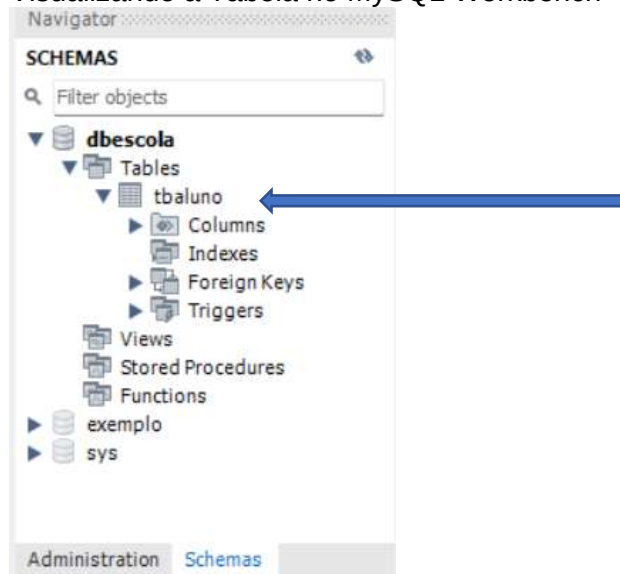
Importante: neste momento estamos criando apenas a **estrutura** da tabela. Ainda não estamos cadastrando alunos.

Os dados serão inseridos posteriormente, quando estudarmos **DML**.

Essa distinção é importante:

DDL cria a estrutura. DML trabalhará com os dados armazenados nessa estrutura.

Visualizando a Tabela no MySQL Workbench



SHOW TABLES – Visualizando as Tabelas

Também podemos utilizar um comando para verificar quais tabelas existem no banco de dados selecionado.

```
SHOW TABLES;
```

Considerando nosso exemplo, teremos um resultado semelhante a:

Tables_in_dbescola
tbaluno

Assim podemos confirmar que a tabela foi criada.

A documentação do MySQL confirma que `SHOW TABLES` lista as tabelas não temporárias do banco especificado; ele também pode listar views, assunto que estudaremos posteriormente.

Atenção

Assim como `SHOW DATABASES`, `SHOW TABLES` pertence à família de instruções `SHOW` disponibilizada pelo MySQL. Portanto, não devemos apresentar sua sintaxe como se fosse necessariamente igual em outros SGBDs.

Fácil Assim!

`SHOW DATABASES` → mostra os bancos.

`SHOW TABLES` → mostra as tabelas.

DESCRIBE – Visualizando a Estrutura da Tabela

Já sabemos verificar se a tabela existe.

Mas e se quisermos descobrir **quais colunas existem dentro dela?**

No MySQL podemos utilizar:

- `DESCRIBE <nome_tabela>;`

Por exemplo:

```
DESCRIBE tbAluno;
```

Também podemos encontrar a forma abreviada:

```
desc tbAluno;
```

No MySQL, `DESCRIBE` e `DESC` são aceitos para essa finalidade. A documentação também informa que `DESCRIBE` é sinônimo de `EXPLAIN`, embora na prática `DESCRIBE` seja frequentemente utilizado para obter informações sobre a estrutura de uma tabela.

Resultado esperado

O MySQL apresentará informações semelhantes a estas:

Field	Type	Null	Key	Default	Extra
AlunoID	int	YES		NULL	
Nome	varchar(60)	YES		NULL	
DataNascimento	date	YES		NULL	
Estado	char(2)	YES		NULL	

Não precisamos entender todas essas colunas agora.

Neste momento observe principalmente:

- **Field** → mostra o nome da coluna.
- **Type** → mostra o tipo de dado da coluna.

Por exemplo:

Field	Type
AlunoID	int
Nome	varchar(60)
DataNascimento	date
Estado	char(2)

Mais adiante, quando estudarmos as **restrições**, entenderemos melhor informações como Null, Key, Default e Extra.

CREATE TABLE, SHOW TABLES e DESCRIBE

Agora já conseguimos realizar um pequeno ciclo completo:

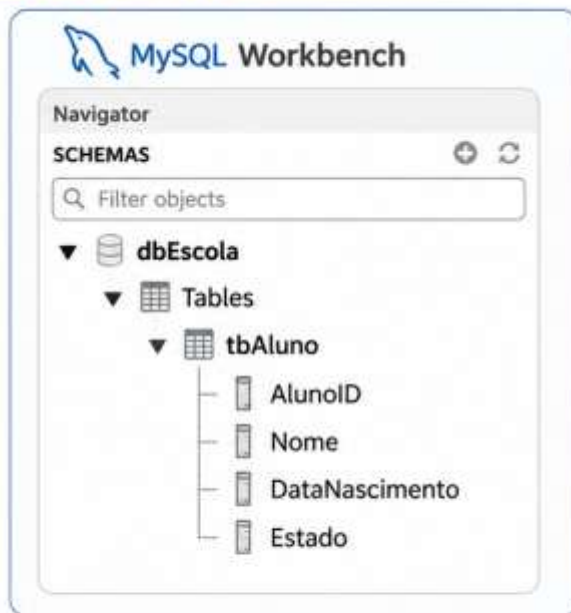
```
USE dbEscola;  
CREATE TABLE tbAluno (  
    AlunoID INT,  
    Nome VARCHAR(60),  
    DataNascimento DATE,  
    Estado CHAR(2)  
);  
SHOW TABLES;  
DESCRIBE tbAluno;
```

Podemos ler essa sequência assim:

- **USE** → seleciona o banco.
- **CREATE TABLE** → cria a tabela.
- **SHOW TABLES** → verifica se a tabela existe.
- **DESCRIBE** → mostra sua estrutura.

RESUMO

DO COMANDO ATÉ A TABELA



FÁCIL ASSIM!

- ➔ **CREATE TABLE** cria a tabela.
- ➔ **SHOW TABLES** mostra as tabelas existentes.
- ➔ **DESCRIBE** mostra a estrutura de uma tabela.

ALTER TABLE – Alterando uma Tabela

Até agora aprendemos a criar uma tabela utilizando `CREATE TABLE`.

Por exemplo:

```
CREATE TABLE tbAluno (  
    AlunoID INT,  
    Nome VARCHAR(60),  
    DataNascimento DATE,  
    Estado CHAR(2)  
);
```

Mas imagine que a tabela já foi criada e percebemos que precisamos fazer alguma alteração em sua estrutura.

Podemos precisar:

- adicionar uma nova coluna;
- modificar uma coluna existente;
- excluir uma coluna;
- renomear uma coluna.

Para essas situações podemos utilizar o comando:

`ALTER TABLE`

A palavra **ALTER** pode ser entendida como **alterar ou modificar**.

Portanto:

ALTER TABLE é utilizado para modificar a estrutura de uma tabela existente.

Estrutura do ALTER TABLE

Como `ALTER TABLE` pode realizar diferentes alterações, sua sintaxe muda um pouco de acordo com aquilo que desejamos fazer.

Vamos aprender inicialmente quatro operações:

Operação	O que faz
ADD	adiciona uma coluna
MODIFY	modifica a definição de uma coluna
DROP	exclui uma coluna
RENAME COLUMN	renomeia uma coluna

Vamos estudar cada uma separadamente.

ADD – Adicionando uma Coluna

Imagine que criamos a tabela `tbAluno`, mas esquecemos de incluir o telefone do aluno. Não precisamos excluir a tabela e criá-la novamente.

Podemos adicionar uma nova coluna.

Sintaxe

```
ALTER TABLE <nome_tabela>  
ADD <nome_coluna> <tipo>;
```

Vamos entender:

- **ALTER** → indica que desejamos alterar um objeto.
- **TABLE** → indica que o objeto alterado será uma tabela.
- **<nome_tabela>** → nome da tabela que será alterada.
- **ADD** → indica que desejamos adicionar algo.
- **<nome_coluna>** → nome da nova coluna.
- **<tipo>** → tipo de dado da nova coluna.

Exemplo

Vamos adicionar a coluna `Telefone`:

```
ALTER TABLE tbAluno  
ADD Telefone VARCHAR(20);
```

Podemos ler o comando como:

“Altere a tabela `tbAluno` adicionando a coluna `Telefone` do tipo `VARCHAR(20)`.”

Depois da alteração, podemos conferir a estrutura:

```
DESCRIBE tbAluno;
```

Resultado esperado

Antes:

Field	Type	Null	Key	Default	Extra
AlunoID	int	YES		NULL	
Nome	varchar(60)	YES		NULL	
DataNascimento	date	YES		NULL	
Estado	char(2)	YES		NULL	

Depois:

Field	Type	Null	Key	Default	Extra
AlunoID	int	YES		NULL	
Nome	varchar(60)	YES		NULL	
DataNascimento	date	YES		NULL	
Estado	char(2)	YES		NULL	
Telefone	varchar(20)	YES		NULL	

O MySQL permite escrever `ADD` ou `ADD COLUMN`. Para manter nossos primeiros exemplos mais simples, utilizaremos:

```
ADD Telefone VARCHAR (20) ;
```

As duas formas são válidas no MySQL.

MODIFY – Modificando uma Coluna

Também podemos modificar a definição de uma coluna que já existe.

Imagine que a coluna `Nome` foi criada assim:

```
Nome VARCHAR (60)
```

Depois percebemos que precisamos permitir nomes maiores.

Podemos modificar a coluna para:

```
Nome VARCHAR (100)
```

Sintaxe

```
ALTER TABLE <nome_tabela>  
MODIFY <nome_coluna> <nova_definicao>;
```

Exemplo

```
ALTER TABLE tbAluno  
MODIFY Nome VARCHAR(100);
```

Podemos ler:

“Altere a tabela tbAluno modificando a coluna Nome para VARCHAR(100).”

Depois:

```
DESCRIBE tbAluno;
```

Agora encontraremos:

Field	Type	Null	Key	Default	Extra
AlunoID	int	YES		NULL	
Nome	varchar(100)	YES		NULL	
DataNascimento	date	YES		NULL	
Estado	char(2)	YES		NULL	
Telefone	varchar(20)	YES		NULL	



MODIFY pode alterar a definição da coluna, inclusive seu tipo de dado. Entretanto, quando uma tabela já possui dados, uma alteração incompatível pode não ser possível ou pode exigir conversão dos valores existentes. O próprio MySQL trata mudanças de tipo como operações que podem exigir reorganização da tabela.

MODIFY = modificar a definição da coluna.

DROP COLUMN – Excluindo uma Coluna

Também podemos remover uma coluna que não é mais necessária.

Sintaxe

```
ALTER TABLE <nome_tabela>  
DROP COLUMN <nome_coluna>;
```

Exemplo

Vamos excluir a coluna `Telefone`:

```
ALTER TABLE tbAluno  
DROP COLUMN Telefone;
```

Podemos ler:

“Altere a tabela tbAluno excluindo a coluna Telefone.”

Depois podemos conferir:

```
DESCRIBE tbAluno;
```

A coluna `Telefone` não aparecerá mais na estrutura.

Atenção

Ao excluir uma coluna, os dados que estavam armazenados nela também são removidos. Portanto, `DROP COLUMN` deve ser utilizado com cuidado.

E aqui vale reforçar uma diferença importante:

- `DROP DATABASE` → exclui o banco
- `DROP TABLE` → exclui a tabela
- `DROP COLUMN` → exclui uma coluna

O significado de `DROP` continua sendo **excluir**. O objeto informado determina **o que será excluído**.

RENAME COLUMN – Renomeando uma Coluna

Também podemos alterar somente o **nome** de uma coluna. Imagine que temos:

Estado

e desejamos mudar para:

UF

Sintaxe

```
ALTER TABLE <nome_tabela>
```

```
RENAME COLUMN <nome_atual> TO <novo_nome>;
```

Exemplo

```
ALTER TABLE tbAluno  
RENAME COLUMN Estado TO UF;
```

Vamos entender:

- **ALTER TABLE tbAluno** → altera a tabela `tbAluno`.
- **RENAME COLUMN** → indica que uma coluna será renomeada.
- **Estado** → nome atual.
- **TO** → indica o novo nome.
- **UF** → novo nome da coluna.

Podemos ler:

“Altere a tabela `tbAluno` renomeando a coluna `Estado` para `UF`.”

Depois:

```
DESCRIBE tbAluno;
```

Teremos:

Field	Type	Null	Key	Default	Extra
AlunoID	int	YES		NULL	
Nome	varchar(100)	YES		NULL	
DataNascimento	date	YES		NULL	
UF	char(2)	YES		NULL	
Telefone	varchar(20)	YES		NULL	

Observe que:

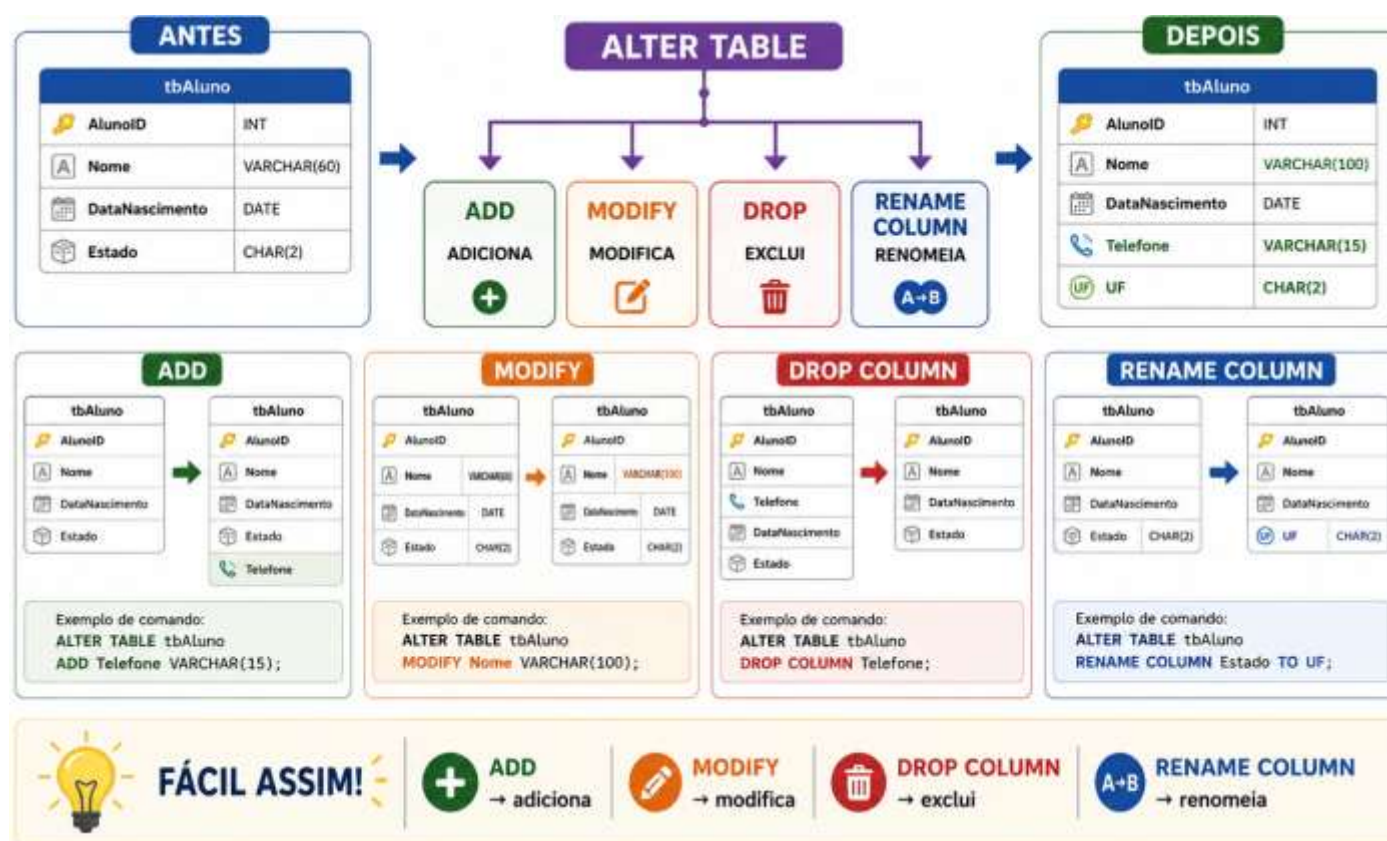
Estado CHAR(2)

passou a ser:

UF CHAR(2)

O nome mudou, mas o tipo permaneceu `CHAR(2)`.

RESUMO



SQL Constraints – Restrições

Até agora aprendemos a criar uma tabela informando o nome das colunas e seus tipos de dados.

Por exemplo:

```
CREATE TABLE tbAluno (  
    AlunoID INT,  
    Nome VARCHAR(100),  
    DataNascimento DATE,  
    UF CHAR(2)  
);
```

Porém, em algumas situações, apenas definir o tipo de dado não é suficiente.

Imagine algumas regras:

- O aluno pode ficar sem nome?
- Dois alunos podem possuir o mesmo código?
- Uma idade pode ser negativa?
- Como podemos relacionar uma tabela com outra?

Para controlar situações como essas, podemos utilizar **restrições**.

Em SQL, essas restrições também são chamadas de:

Constraints

Uma **constraint** é uma regra aplicada à estrutura da tabela para ajudar a controlar os valores permitidos e manter a consistência dos dados.

No MySQL, restrições podem fazer parte da definição das colunas ou da própria tabela. `PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY` e `CHECK`, por exemplo, aparecem oficialmente entre os tipos de constraints do MySQL.

Inicialmente estudaremos:

Restrição	Finalidade
<code>NOT NULL</code>	não permite valor <code>NULL</code>
<code>UNIQUE</code>	não permite valores repetidos
<code>DEFAULT</code>	define um valor padrão
<code>CHECK</code>	estabelece uma condição para os valores
<code>PRIMARY KEY</code>	identifica cada registro
<code>FOREIGN KEY</code>	estabelece relacionamento entre tabelas
<code>AUTO_INCREMENT</code>	gera valores numéricos automaticamente

Vamos aprender uma de cada vez.

NOT NULL

Quando criamos uma coluna normalmente, ela pode permitir a ausência de um valor.

Essa ausência é representada em SQL por:

NULL

`NULL` representa a **ausência de um valor**.

Não devemos pensar em `NULL` como número zero ou como um texto vazio. Ele indica que não há um valor armazenado naquela posição.

Imagine que todo aluno obrigatoriamente deve possuir um nome.

Podemos definir:

```
Nome VARCHAR(100) NOT NULL
```

A restrição:

`NOT NULL`

indica que essa coluna **não poderá receber `NULL`**.

Exemplo

```
CREATE TABLE tbAluno (  
    AlunoID INT,  
    Nome VARCHAR(100) NOT NULL,  
    DataNascimento DATE,  
    UF CHAR(2)  
);
```

Podemos ler:

“A coluna Nome armazena um texto de até 100 caracteres e não permite valor `NULL`.”

Visualizando com DESCRIBE
`DESCRIBE tbAluno;`

Teremos um resultado semelhante a:

Field	Type	Null	Key	Default	Extra
AlunoID	int	YES		NULL	
Nome	varchar(100)	NO		NULL	
DataNascimento	date	YES		NULL	
UF	char(2)	YES		NULL	

Agora já conseguimos entender uma das informações apresentadas pelo `DESCRIBE`:

Null = NO → a coluna não aceita `NULL`.

Fácil Assim!

NOT NULL = o valor não pode ficar ausente.

UNIQUE

Imagine agora que cada aluno tenha um endereço de e-mail e que não desejamos cadastrar o mesmo e-mail para dois alunos diferentes.

Podemos utilizar:

UNIQUE

A restrição `UNIQUE` exige que os valores definidos por essa chave sejam distintos. Violações de uma chave `UNIQUE` normalmente geram erro no MySQL.

Exemplo

```
CREATE TABLE tbAluno (  
    AlunoID INT,  
    Nome VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    DataNascimento DATE,  
    UF CHAR(2)  
);
```

Neste caso:

`Email VARCHAR(100) UNIQUE`

indica que valores de e-mail repetidos não serão permitidos pela restrição.

Por exemplo:

carlos@email.com	✓
maria@email.com	✓
carlos@email.com	X repetido

Neste momento não precisamos inserir esses registros. O objetivo é compreender **a regra definida na estrutura**.

Fácil Assim!

UNIQUE = não permite valores repetidos.

DEFAULT

Em algumas situações queremos definir um valor que será utilizado quando nenhum valor for informado para determinada coluna.

Para isso podemos utilizar:
DEFAULT

Imagine que os alunos cadastrados sejam considerados ativos inicialmente.

Podemos criar:

```
Ativo BOOLEAN DEFAULT TRUE
```

Isso define `TRUE` como valor padrão da coluna.

Outro exemplo simples:

```
UF CHAR(2) DEFAULT 'PR'
```

Nesse caso, quando nenhum valor for fornecido para `UF` em uma inserção que utilize o padrão da coluna, o

MySQL poderá utilizar:

```
PR
```

Exemplo na tabela

```
CREATE TABLE tbAluno (  
    AlunoID INT,  
    Nome VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    DataNascimento DATE,  
    UF CHAR(2) DEFAULT 'PR'  
);
```

Observe:

```
UF CHAR(2) DEFAULT 'PR'
```

- **UF** → nome da coluna
- **CHAR(2)** → tipo
- **DEFAULT 'PR'** → valor padrão

Fácil Assim!

DEFAULT = valor padrão.

CHECK

Às vezes precisamos estabelecer uma condição para os valores armazenados.

Imagine uma coluna:

```
Idade INT
```

Tecnicamente, `INT` permite números negativos.

Mas faria sentido cadastrar:

```
Idade = -15
```

Não.

Podemos estabelecer uma condição utilizando:
CHECK

Exemplo

```
Idade INT CHECK (Idade >= 0)
```

A condição é:

```
Idade >= 0
```

Ou seja:

“A idade deve ser maior ou igual a zero.”

No MySQL atual, constraints **CHECK** são efetivamente avaliadas quando operações como **INSERT** e **UPDATE** tentam fornecer valores; se a expressão resultar em **FALSE**, ocorre erro.

Isso é importante porque materiais antigos sobre MySQL podem afirmar que **CHECK** é apenas aceito e ignorado. **Isso não representa o comportamento do MySQL 8.x atual.**

Exemplo

```
CREATE TABLE tbAluno (  
    AlunoID INT,  
    Nome VARCHAR(100) NOT NULL,  
    Idade INT CHECK (Idade >= 0)  
);
```

Visualmente:

Idade = 18	✓
Idade = 25	✓
Idade = 0	✓
Idade = -5	X

Fácil Assim!

CHECK = verifica uma condição.

PRIMARY KEY – Chave Primária

Agora chegamos a uma das restrições mais importantes dos bancos de dados relacionais.

Lembre-se da tabela apresentada no início:

AlunoID	Nome
1	Carlos
2	Maria
3	João

Como podemos identificar **exatamente um aluno**?

O nome não é uma boa escolha, porque duas pessoas podem possuir o mesmo nome.

Por isso podemos utilizar um código:

```
AlunoID
```

Esse código pode ser definido como:

Chave Primária — Primary Key

A **PRIMARY KEY** identifica cada registro da tabela por meio de um valor único e não permite **NULL**. O MySQL associa também um índice à chave primária.

Sintaxe simples

AlunoID INT PRIMARY KEY

Exemplo

```
CREATE TABLE tbAluno (  
    AlunoID INT PRIMARY KEY,  
    Nome VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    DataNascimento DATE,  
    UF CHAR(2) DEFAULT 'PR'  
);
```

Agora:

AlunoID

↓

PRIMARY KEY

↓

identifica cada aluno

Teríamos:

1 → Carlos

2 → Maria

3 → João

Não poderíamos utilizar novamente:

1 → Pedro X

porque o valor 1 já identifica Carlos.

Chave Primária Composta

Até agora utilizamos uma única coluna como chave primária.

Por exemplo:

```
CREATE TABLE aluno (  
    id_aluno INT PRIMARY KEY,  
    nome VARCHAR(100)  
);
```

Nesse caso, id_aluno sozinho identifica cada registro da tabela.

Porém, em alguns projetos, **a identificação de um registro pode depender da combinação de duas ou mais colunas.**

Quando isso acontece, podemos utilizar uma **chave primária composta.**

Exemplo

Imagine uma tabela que registra quais alunos estão matriculados em quais turmas:

id_aluno	id_turma
1	10
1	20
2	10
2	30

Observe que id_aluno sozinho não identifica um registro, pois o mesmo aluno pode participar de várias turmas.

Da mesma forma, `id_turma` sozinho também não identifica um registro, pois uma turma pode possuir vários alunos.

Porém, a combinação:

`id_aluno + id_turma`

identifica cada matrícula.

Nesse caso, podemos definir as duas colunas como uma chave primária composta:

```
CREATE TABLE matricula (  
    id_aluno INT,  
    id_turma INT,  
    PRIMARY KEY (id_aluno, id_turma)  
);
```

Vamos entender

`PRIMARY KEY (id_aluno, id_turma)`

indica que a chave primária é formada pela **combinação das duas colunas**.

Assim:

1 | 10

1 | 20

2 | 10

podem existir.

Mas:

1 | 10

1 | 10

não poderá ocorrer, pois a mesma combinação estaria sendo repetida.

Fácil Assim!

Chave primária simples → formada por uma coluna.

Chave primária composta → formada pela combinação de duas ou mais colunas.

PRIMARY KEY x UNIQUE

As duas trabalham com unicidade, mas possuem finalidades diferentes.

PRIMARY KEY	UNIQUE
identifica os registros da tabela	garante unicidade
não aceita NULL	possui regras próprias para NULL
uma chave primária por tabela	podem existir várias chaves UNIQUE

Fácil Assim!

PRIMARY KEY = identificação única do registro.

AUTO_INCREMENT

Observe nossa chave:

```
AlunoID INT PRIMARY KEY
```

Quando cadastrarmos alunos, precisaremos de identificadores como:

```
1  
2  
3  
4  
5
```

O MySQL possui um recurso muito útil para gerar esses valores automaticamente:

```
AUTO_INCREMENT
```

Exemplo:

```
AlunoID INT AUTO_INCREMENT PRIMARY KEY
```

Assim, novos registros podem receber automaticamente números sequenciais quando não fornecemos um valor para essa coluna. A documentação oficial utiliza justamente `AUTO_INCREMENT` para gerar uma identidade única para novas linhas.

Por exemplo:

```
Carlos → 1  
Maria → 2  
João → 3
```

O MySQL gerencia a geração desses valores.

Exemplo

```
CREATE TABLE tbAluno (  
    AlunoID INT AUTO_INCREMENT PRIMARY KEY,  
    Nome VARCHAR(100) NOT NULL,  
    Email VARCHAR(100) UNIQUE,  
    DataNascimento DATE,  
    UF CHAR(2) DEFAULT 'PR'  
);
```

AUTO_INCREMENT = gera automaticamente o próximo valor numérico da sequência.

Observação: `AUTO_INCREMENT` não é uma constraint. É um atributo do MySQL utilizado para gerar automaticamente valores numéricos sequenciais.

FOREIGN KEY – Chave Estrangeira

Relacionamento entre tabelas.

Imagine duas tabelas:

CLIENTE	PEDIDO
ClienteID	PedidoID
Nome	DataPedido
	ClienteID

Um cliente pode realizar pedidos.

Precisamos então estabelecer uma ligação entre:

CLIENTE	PEDIDO
ClienteID	ClienteID

Para isso utilizamos uma:

FOREIGN KEY — Chave Estrangeira

No MySQL, uma chave estrangeira permite relacionar dados entre tabelas e sua constraint ajuda a manter os dados relacionados consistentes.

Primeiro criamos a tabela Cliente

```
CREATE TABLE tbCliente (  
    ClienteID INT AUTO_INCREMENT PRIMARY KEY,  
    Nome VARCHAR(100) NOT NULL  
);
```

Temos:

tbCliente
ClienteID (PK)
Nome

Agora podemos criar a tabela de pedidos.

```
CREATE TABLE tbPedido (  
    PedidoID INT AUTO_INCREMENT PRIMARY KEY,  
    DataPedido DATE,  
    ClienteID INT,  
    FOREIGN KEY (ClienteID)  
        REFERENCES tbCliente(ClienteID)  
);
```

Vamos entender esta parte:

```
FOREIGN KEY (ClienteID)  
REFERENCES tbCliente(ClienteID)
```

FOREIGN KEY → indica a criação da chave estrangeira.

(ClienteID) → coluna da tabela tbPedido que será utilizada no relacionamento.

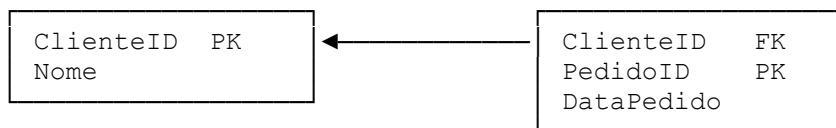
REFERENCES → indica qual tabela e coluna serão referenciadas.

tbCliente(ClienteID) → tabela e coluna de referência.

Podemos visualizar:

tbCliente

tbPedido



Agora aquelas siglas começam a fazer sentido:

- **PK** → Primary Key → Chave Primária
- **FK** → Foreign Key → Chave Estrangeira

A tabela que contém o valor referenciado é frequentemente chamada de **tabela pai**, e a tabela que possui a chave estrangeira é a **tabela filha**. A documentação do MySQL utiliza exatamente essa relação parent/child.

Por que isso é importante?

Imagine que exista:

tbCliente

```
ClienteID | Nome
1         | Carlos
2         | Maria
```

Um pedido com:

ClienteID = 1

estará relacionado a Carlos.

Outro com:

ClienteID = 2

estará relacionado a Maria.

É esse relacionamento que permite organizar informações em tabelas diferentes mantendo uma ligação entre elas.

Fácil Assim!

PRIMARY KEY

FOREIGN KEY

identifica

referencia

└──────────┴──────────┘
relacionamento

PK identifica. FK relaciona.

Juntando as Restrições

Agora já conseguimos criar uma tabela muito mais completa:

```
CREATE TABLE tbAluno (  
  AlunoID INT AUTO_INCREMENT PRIMARY KEY,  
  Nome VARCHAR(100) NOT NULL,  
  Email VARCHAR(100) UNIQUE,  
  DataNascimento DATE,  
  UF CHAR(2) DEFAULT 'PR',  
  Idade INT CHECK (Idade >= 0)  
);
```

Observe como cada parte possui uma função:

```
AlunoID  
INT  
AUTO_INCREMENT  
PRIMARY KEY
```

→ número inteiro, gerado automaticamente e utilizado como identificação.

```
Nome  
VARCHAR(100)  
NOT NULL
```

→ texto obrigatório.

```
Email  
VARCHAR(100)  
UNIQUE
```

→ e-mail com valor não repetido pela chave `UNIQUE`.

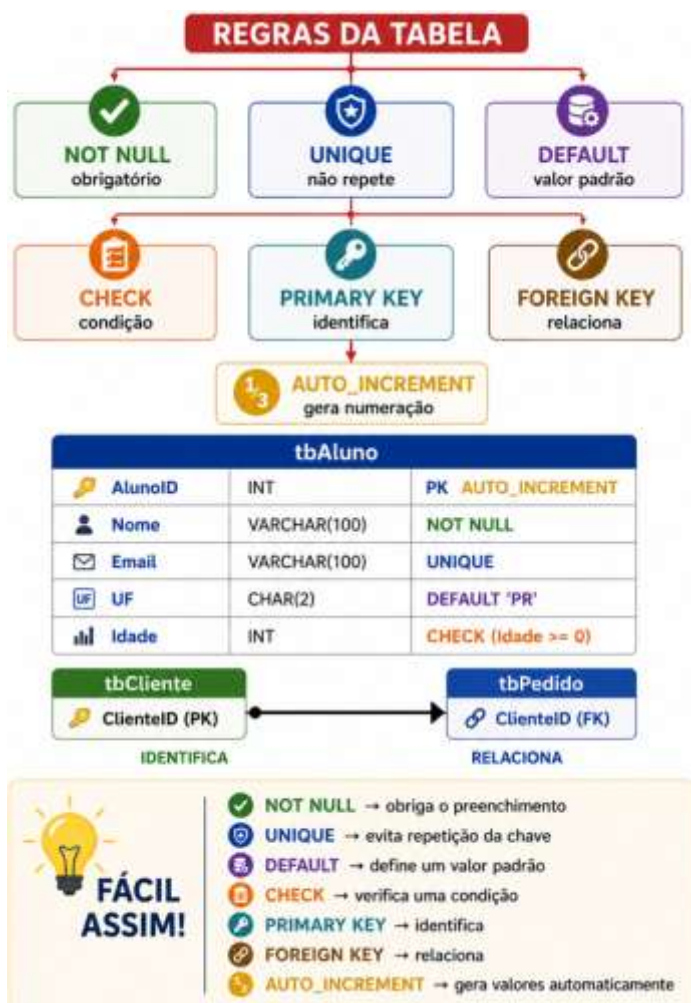
```
UF  
CHAR(2)  
DEFAULT 'PR'
```

→ utiliza `PR` como valor padrão quando aplicável.

```
Idade  
INT  
CHECK (Idade >= 0)
```

→ não permite que a condição seja falsa.

RESUMO



DROP TABLE – Excluindo uma Tabela

Já utilizamos a palavra `DROP` anteriormente:

```
DROP DATABASE dbEscola;
```

Naquele caso, estávamos excluindo um **banco de dados**.

Também vimos:

```
ALTER TABLE tbAluno
```

```
DROP COLUMN Telefone;
```

Naquele caso, estávamos excluindo um **banco de dados**.

Também vimos:

```
ALTER TABLE tbAluno
```

```
DROP COLUMN Telefone;
```

Nesse caso, estávamos excluindo uma **coluna**.

Agora utilizaremos `DROP` para excluir uma **tabela inteira**.

Sintaxe

```
DROP TABLE <nome_tabela>;
```

Vamos entender:

- **DROP** → comando utilizado para excluir um objeto.
- **TABLE** → indica que o objeto que será excluído é uma tabela.
- **<nome_tabela>** → nome da tabela que desejamos excluir.

Exemplo

Imagine uma tabela utilizada apenas para testes:

`tbTeste`

Para excluí-la:

```
DROP TABLE tbTeste;
```

Após a execução, a tabela deixa de existir.

E aqui precisamos ter bastante atenção:

- **DROP TABLE exclui a estrutura da tabela e os dados armazenados nela.**

Podemos verificar:

```
SHOW TABLES;
```

A tabela excluída não aparecerá mais.

DROP TABLE IF EXISTS

Existe também a possibilidade de verificar a existência da tabela antes de tentar excluí-la.

```
DROP TABLE IF EXISTS tbTeste;
```

Já conhecemos:

```
IF EXISTS
```

do comando:

```
DROP DATABASE IF EXISTS dbEscola;
```

A ideia é a mesma:

“Exclua a tabela tbTeste se ela existir.”

Se a tabela não existir, `IF EXISTS` evita que a instrução falhe por esse motivo; o MySQL gera uma informação diagnóstica em vez do erro correspondente à inexistência da tabela.

Fácil Assim!

`DROP DATABASE` → exclui um banco
`DROP TABLE` → exclui uma tabela
`DROP COLUMN` → exclui uma coluna

DROP = excluir.

O restante do comando informa **o que será excluído**.

TRUNCATE TABLE – Limpando uma Tabela

Agora precisamos diferenciar duas situações.

Imagine uma tabela com vários registros:

AlunoID	Nome
1	Carlos
2	Maria
3	João

Em determinado momento queremos **remover todos os registros**, mas continuar utilizando a tabela.

Ou seja:

Queremos apagar os dados...
mas NÃO queremos apagar a tabela.

Para isso existe:

`TRUNCATE TABLE`

Sintaxe

`TRUNCATE TABLE <nome_tabela>;`

Exemplo

```
TRUNCATE TABLE tbAluno;
```

Depois da execução:

ANTES

tbAluno

AlunoID	Nome
1	Carlos
2	Maria
3	João

DEPOIS

tbAluno

AlunoID	Nome
---------	------

A TABELA CONTINUA EXISTINDO.

Portanto:

`TRUNCATE TABLE` esvazia completamente a tabela, mantendo sua estrutura.

No MySQL, `TRUNCATE TABLE` é classificado como **DDL**, apesar de seu efeito sobre os registros ser semelhante a excluir todas as linhas.

E o `AUTO_INCREMENT`?

Imagine que `AlunoID` tenha:

`AUTO_INCREMENT`

e os valores já tenham chegado a:

1
2
3
4
5

No MySQL, `TRUNCATE TABLE` reinicia a sequência `AUTO_INCREMENT` para o valor inicial.

Atenção

`TRUNCATE TABLE` remove **todos** os registros.

Não utilizamos:

`TRUNCATE TABLE tbAluno WHERE ...;`

Essa sintaxe não existe.

Com `TRUNCATE`, a tabela é esvaziada completamente.

DROP TABLE x TRUNCATE TABLE

DROP TABLE

`DROP TABLE tbAluno;`

Resultado:

DADOS → excluídos

ESTRUTURA → excluída

A tabela deixa de existir.

TRUNCATE TABLE

`TRUNCATE TABLE tbAluno;`

Resultado:

DADOS → removidos

ESTRUTURA → permanece

A tabela continua existindo.

Fácil Assim!

`DROP TABLE`

↓

 joga a tabela inteira fora

`TRUNCATE TABLE`

↓

 esvazia a tabela

RENAME TABLE – Renomeando uma Tabela

Anteriormente aprendemos a renomear uma **coluna**:

```
ALTER TABLE tbAluno  
RENAME COLUMN Estado TO UF;
```

Também podemos renomear uma tabela inteira.

Para isso o MySQL possui:

```
RENAME TABLE
```

Sintaxe

```
RENAME TABLE <nome_atual>  
TO <novo_nome>;
```

Exemplo

Imagine que temos:

tbAluno

e desejamos mudar para:

tbEstudante

Utilizamos:

```
RENAME TABLE tbAluno  
TO tbEstudante;
```

Podemos ler:

“Renomeie a tabela tbAluno para tbEstudante.”

Depois podemos executar:

```
SHOW TABLES;
```

Agora encontraremos:

tbEstudante

em vez de:

tbAluno

A sintaxe está de acordo com o MySQL 8.x. O MySQL também permite renomear mais de uma tabela na mesma instrução, mas não precisamos dessa possibilidade neste momento.

Não confundir

Renomear tabela:

```
RENAME TABLE tbAluno  
TO tbEstudante;
```

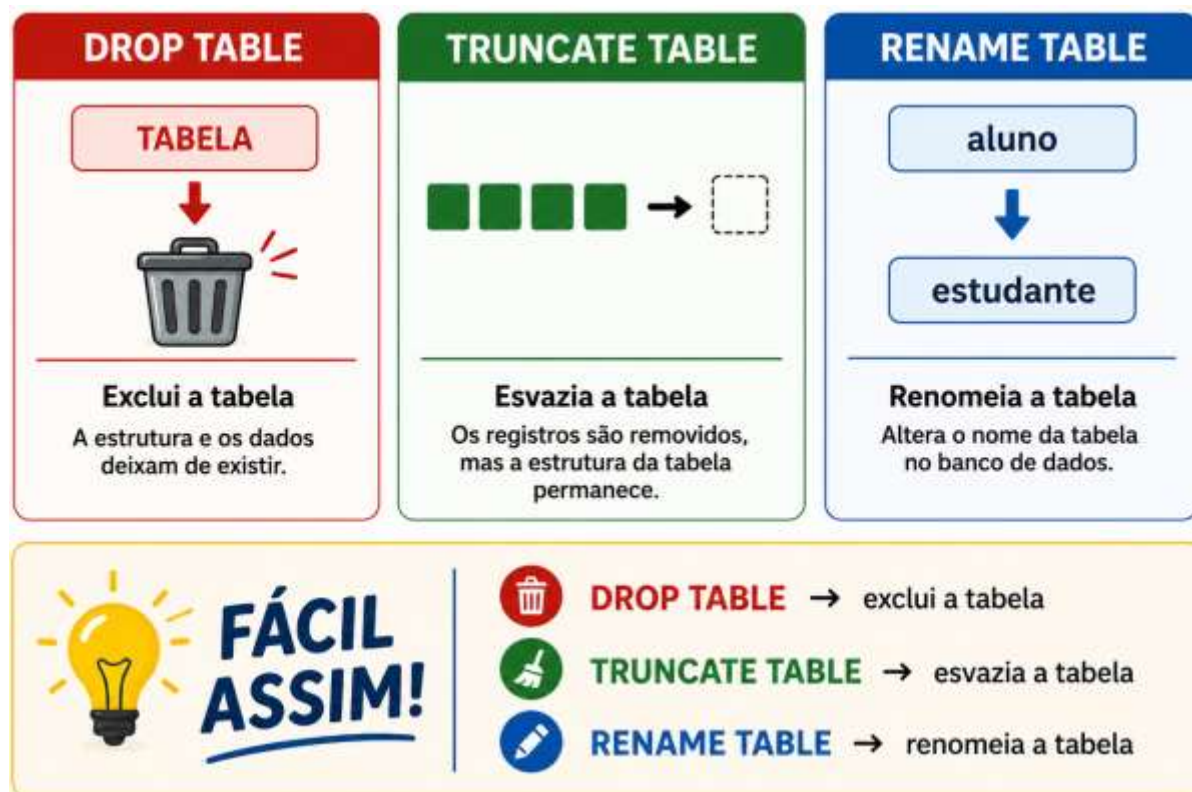
Renomear coluna:

```
ALTER TABLE tbAluno  
RENAME COLUMN Estado TO UF;
```

Fácil Assim!



RESUMO



DO PROJETO AO BANCO DE DADOS FÍSICO

Antes de iniciarmos os exercícios, vamos relacionar esta etapa com aquilo que já estudamos em Projeto de Banco de Dados.

Um banco de dados não deve ser criado a partir de tabelas escolhidas aleatoriamente. Primeiro conhecemos o problema e realizamos o levantamento das necessidades e das regras de negócio.

A partir dessas informações construímos o modelo conceitual, transformamos esse modelo em um modelo lógico e aplicamos a normalização quando necessário.

Agora utilizaremos a linguagem SQL para transformar esse projeto em estruturas reais dentro do MySQL. Portanto, a implementação física **não substitui o projeto**. Ela é a continuação dele.

No projeto	Na implementação física
Entidade	Tabela
Atributo	Coluna
Identificador	PRIMARY KEY
Atributo obrigatório	NOT NULL
Valor não repetido	UNIQUE
Regra sobre valores	CHECK
Valor inicial	DEFAULT
Relacionamento	FOREIGN KEY
Relação associativa	Tabela associativa
Domínio	Tipo de dado

EXERCÍCIOS PRÁTICOS

Do Projeto de Banco de Dados à Implementação Física no MySQL

Antes de começar

Até este momento, você aprendeu a **projetar um banco de dados**.

Durante o levantamento de requisitos, procuramos compreender as necessidades do cliente e identificar suas regras de negócio.

No modelo conceitual, representamos entidades, atributos, relacionamentos e cardinalidades.

No modelo lógico, transformamos esse projeto em relações, identificamos chaves e organizamos a estrutura.

Durante a normalização, analisamos essas relações para reduzir redundâncias e evitar problemas de armazenamento.

Agora começa uma nova etapa:

Transformar o projeto em um banco de dados real dentro do SGBD.

No MySQL, utilizaremos a linguagem SQL para realizar essa implementação.

Do projeto para o banco físico

No projeto	Na implementação física
Entidade	Tabela
Atributo	Coluna
Identificador	PRIMARY KEY
Atributo obrigatório	NOT NULL
Valor que não pode se repetir	UNIQUE
Regra sobre valores permitidos	CHECK
Valor inicial definido pelo negócio	DEFAULT
Relacionamento	FOREIGN KEY
Relação associativa	Tabela associativa
Domínio do atributo	Tipo de dado

Orientações

Os exercícios apresentam **necessidades, projetos e regras de negócio**.

Nem sempre será informado qual comando SQL, tipo de dado ou restrição deverá ser utilizado.

Você deverá interpretar o problema e decidir como representá-lo no MySQL.

Os exercícios são cumulativos. Portanto, conhecimentos utilizados anteriormente poderão ser necessários novamente.

ETAPA 1 — FIXAÇÃO DOS PRIMEIROS COMANDOS

Exercício Prático 01 – Preparando os Ambientes

Contexto Empresarial

Uma empresa de desenvolvimento recebeu quatro novos projetos:

- clínica veterinária;
- oficina mecânica;
- escola de idiomas;
- loja de materiais de construção.

As equipes responsáveis já iniciaram o levantamento e a modelagem. Agora será necessário preparar os ambientes onde futuramente esses projetos serão implementados.

Tarefa

Prepare um banco de dados independente para cada projeto:

- dbVeterinaria
- dbOficina
- dbEscolalDomas
- dbMateriais

Depois, verifique no servidor se os quatro ambientes foram realmente criados.
Deixe o projeto da oficina preparado para receber os próximos comandos.

Regras obrigatórias

O script deverá possuir um comentário identificando a atividade.
Todas as operações deverão ser realizadas por SQL.

Exercício Prático 02 – Um Novo Projeto

Contexto Empresarial

A empresa recebeu um quinto cliente.
Uma academia necessita de um sistema para controlar alunos, planos e matrículas.
A equipe de modelagem iniciou o projeto e solicitou a preparação do ambiente físico.

Tarefa

Prepare um banco chamado:

- dbAcademia

Confirme sua criação juntamente com os bancos anteriores.
Deixe esse novo ambiente preparado para utilização.

Regra obrigatória

Tente resolver a atividade lembrando dos comandos utilizados anteriormente antes de consultar exercícios anteriores.

Exercício Prático 03 – Lendo o Projeto da Oficina

Contexto Empresarial

A equipe de projeto entregou a primeira relação do modelo lógico da oficina.

Projeto recebido — VEICULO

Atributo	Definição no projeto
id_veiculo	Identificador
placa	Até 10 caracteres, obrigatório e único
modelo	Até 100 caracteres, obrigatório
ano	Ano de fabricação
valor_estimado	Valor monetário com duas casas decimais
ativo	Situação do veículo, inicialmente ativo

Tarefa

Transforme essa parte do projeto em uma estrutura física dentro do banco destinado à oficina.
Escolha os tipos de dados e as restrições adequadas.
Depois:

- confira as tabelas existentes;
- confira a estrutura que foi criada.

Regra obrigatória

Implemente o que foi definido no projeto. Não acrescente atributos que não tenham sido solicitados.

Exercício Prático 04 – Projeto x Implementação

Contexto Empresarial

A equipe de modelagem deseja verificar se o projeto foi corretamente implementado.

O modelo aprovado determina:

- id_veiculo identifica cada veículo;
- placa é obrigatória e não pode se repetir;
- modelo é obrigatório;
- ano representa um ano;
- valor_estimado aceita casas decimais;
- ativo possui valor inicial definido pelo projeto.

Tarefa

Consulte a estrutura física criada.

Compare-a com o projeto aprovado.

Caso encontre alguma diferença, faça as alterações necessárias.

Regras obrigatórias

Não exclua a tabela para criá-la novamente.

Antes de qualquer alteração, consulte sua estrutura atual.

Exercício Prático 05 – Mudança nas Regras de Negócio

Contexto Empresarial

Durante uma nova reunião, a oficina solicitou mudanças no projeto.

Projeto atualizado

A estrutura deverá passar a armazenar:

- marca, com até 50 caracteres e preenchimento obrigatório;
- cor, com até 30 caracteres e preenchimento opcional.

Além disso:

- o modelo deverá permitir até 150 caracteres;
- valor_estimado passará a se chamar valor_mercado.

Tarefa

Atualize a implementação física para que ela represente o novo projeto.

Regras obrigatórias

Não recrie a tabela.

Preserve a estrutura existente.

Ao terminar, confira novamente o resultado.

Exercício Prático 06 – Projeto Cancelado

Contexto Empresarial

O projeto da clínica veterinária foi cancelado.

O ambiente criado exclusivamente para ele não deverá permanecer no servidor.

Regra de negócio

O script utilizado pela equipe poderá ser executado mais de uma vez.

Uma nova execução não deverá provocar erro simplesmente porque o banco já tenha sido removido.

Tarefa

Faça a operação necessária e depois confira os bancos disponíveis.

Exercício Prático 07 – Ambientes de Homologação

Contexto Empresarial

Os projetos da oficina e da escola de idiomas entrarão em homologação.

Crie os ambientes:

- dbOficinaHomologacao
- dbEscolaHomologacao

Tarefa

Prepare os dois ambientes.

Confirme que foram criados.

Selecione o ambiente da oficina.

Depois selecione o ambiente da escola.

Após os testes, os dois ambientes deverão ser completamente removidos.

Regra de negócio

A rotina de remoção poderá ser executada novamente mesmo que os ambientes já tenham sido excluídos.

Exercício Prático 08 – Primeira Auditoria

Contexto Empresarial

O coordenador deseja saber o que existe atualmente no servidor.

Tarefa

Utilizando SQL:

- apresente os bancos disponíveis;
- selecione o projeto da oficina;
- apresente suas tabelas;
- apresente a estrutura de veículo;
- selecione o projeto da escola de idiomas;
- apresente suas tabelas;
- retorne ao projeto da oficina.

Regra obrigatória

Nenhuma estrutura deverá ser criada ou excluída.

ETAPA 2 — PROJETO, RELACIONAMENTOS E IMPLEMENTAÇÃO

Exercício Prático 09 – Projeto da Escola de Idiomas

Contexto Empresarial

A equipe responsável pelo projeto da escola concluiu parte do modelo lógico.

CURSO

Atributo	Definição
id_curso	Identificador
nome	Até 100 caracteres, obrigatório e único
carga_horaria	Número inteiro, obrigatório
valor	Valor monetário com duas casas decimais, obrigatório
ativo	Inicialmente ativo

PROFESSOR

Atributo	Definição
id_professor	Identificador
nome	Até 120 caracteres, obrigatório
email	Até 150 caracteres, obrigatório e único
data_admissao	Data obrigatória
ativo	Inicialmente ativo

Tarefa

Implemente as duas relações no banco da escola.
Depois confira as tabelas existentes e suas estruturas.

Regra de negócio

Valores monetários não poderão ser negativos.

Exercício Prático 10 – Do Relacionamento 1:N para o Banco Físico

Contexto Empresarial

No modelo conceitual da escola foi definido:

CURSO 1 : N TURMA

Uma turma pertence a um único curso.
Um curso poderá possuir várias turmas.
No modelo lógico, a equipe definiu:

TURMA

Atributo	Definição
id_turma	Identificador
nome	Até 50 caracteres, obrigatório
data_inicio	Data obrigatória
horario	Horário obrigatório
id_curso	Referência ao curso

Tarefa

Implemente turma.

Regras de negócio

Não poderá existir uma turma associada a um curso inexistente.
O relacionamento definido durante a modelagem deverá ser preservado no banco físico.
Depois confira a estrutura criada.

Exercício Prático 11 – Professor Responsável

Contexto Empresarial

Uma nova regra foi aprovada:

Cada turma deverá possuir um professor responsável.

O professor deverá existir previamente no cadastro.

Tarefa

Atualize o projeto físico existente para representar a nova regra.

Regras obrigatórias

turma já existe.

Não exclua e recrie a tabela.

A nova informação será obrigatória.

Ao terminar, confira a estrutura.

Exercício Prático 12 – Cadastro de Alunos

Projeto lógico — ALUNO

Atributo	Definição
id_aluno	Identificador
nome	Até 150 caracteres, obrigatório
cpf	11 caracteres, obrigatório e único
data_nascimento	Data opcional
email	Até 150 caracteres, único quando informado
data_cadastro	Data e hora
ativo	Inicialmente ativo

Tarefa

Leia o projeto e implemente a relação aluno.

Atenção

CPF possui somente dígitos no projeto atual, mas não será utilizado em operações matemáticas.

Escolha o tipo de dado pensando **na natureza da informação**, e não apenas nos caracteres que ela contém.

Exercício Prático 13 – Do N:N para uma Tabela Associativa

Contexto do Projeto

No modelo conceitual foi identificado:

ALUNO N : N TURMA

Um aluno poderá participar de várias turmas.

Uma turma possuirá vários alunos.

Durante a transformação para o modelo lógico, o relacionamento originou:

MATRICULA

- id_aluno — referência a ALUNO;
- id_turma — referência a TURMA;
- data_matricula — data obrigatória;
- situacao — até 20 caracteres, obrigatória.

Regras de negócio

Um aluno não poderá ser matriculado na mesma turma duas vezes.

Não poderá existir matrícula para aluno inexistente.

Não poderá existir matrícula para turma inexistente.

Tarefa

Implemente a solução definida no modelo lógico.

Atenção

Analise quais atributos, individualmente ou em conjunto, conseguem identificar cada matrícula.

Exercício Prático 14 – Por que Normalizamos?

Contexto

Durante o levantamento inicial, alguém sugeriu armazenar:

ALUNO

id_aluno, nome, curso1, curso2, curso3

Essa estrutura não foi aprovada.

Depois da modelagem e normalização, o projeto resultou em:

ALUNO

CURSO

TURMA

MATRICULA

Tarefa

Observe as estruturas que você já implementou e explique, utilizando um comentário de múltiplas linhas no próprio script, por que não foi necessário criar curso1, curso2 e curso3 dentro de aluno.

Depois, utilize SQL para apresentar as estruturas de aluno e matricula.

Exercício Prático 15 – O Projeto Está Certo, mas o SQL Está?

Contexto

Um desenvolvedor recebeu o seguinte projeto:

CLIENTE

- id_cliente — identificador;
- nome — até 100 caracteres e obrigatório;
- cpf — 11 caracteres, obrigatório e único;
- ativo — inicialmente ativo.

Ele implementou:

```
CREATE TABLE cliente (  
    id_cliente INT,  
    nome VARCHAR(100),  
    cpf CHAR(11),  
    ativo BOOLEAN  
);
```

Tarefa

Compare a implementação com o projeto.

Identifique quais regras do projeto não foram representadas no banco físico.

Depois, utilizando um banco de treinamento, crie a estrutura apresentada e faça as alterações necessárias para que a implementação passe a representar corretamente o projeto.

Regra obrigatória

Depois da criação inicial, as correções deverão ser feitas alterando a estrutura existente.

ETAPA 3 — DESAFIO DE INTEGRAÇÃO

Exercício Prático 16 – Projeto de uma Pequena Locadora

Contexto Empresarial

Uma locadora de equipamentos concluiu o levantamento, o modelo conceitual e o modelo lógico.

Você recebeu o projeto para realizar a implementação física.

Crie um novo ambiente chamado:

dbLocadoraTeste

Projeto lógico

CLIENTE

- identificador;
- nome de até 150 caracteres, obrigatório;
- documento de até 14 caracteres, obrigatório e único;
- telefone opcional;
- situação inicialmente ativa.

CATEGORIA

- identificador;
- nome de até 80 caracteres, obrigatório e único.

EQUIPAMENTO

- identificador;
- nome de até 120 caracteres, obrigatório;
- categoria;
- valor diário com duas casas decimais;
- situação inicialmente disponível.

LOCACAO

- identificador;
- cliente;
- data e hora da retirada;
- data prevista para devolução;
- situação.

ITEM_LOCACAO

- locação;
- equipamento;
- valor diário utilizado na locação.

Regras de negócio

Todo equipamento deverá pertencer a uma categoria existente.

Toda locação deverá pertencer a um cliente existente.

Uma locação poderá possuir vários equipamentos.

Um equipamento poderá participar de várias locações ao longo do tempo.

O mesmo equipamento não poderá aparecer duas vezes na mesma locação.

Valores monetários não poderão ser negativos.

Tarefa

Implemente o projeto no MySQL.

Depois:

- confira o banco criado;
- confira todas as tabelas;
- consulte a estrutura de cada tabela;
- verifique se as chaves e relacionamentos representam o projeto recebido.

Regra obrigatória

Não há indicação de quais comandos, tipos ou restrições deverão ser utilizados.

Você deverá tomar essas decisões a partir do projeto.

Exercício Prático 17 – Mudanças na Locadora

Contexto Empresarial

O cliente solicitou alterações depois da primeira versão.

Novas regras

O nome do equipamento deverá aceitar até 180 caracteres.

Será necessário armazenar a marca do equipamento, com até 80 caracteres e preenchimento obrigatório.

O telefone do cliente deixou de ser necessário.

valor_diario passará a se chamar valor_locacao.

A empresa decidiu que a tabela categoria deverá se chamar tipo_equipamento.

Tarefa

Atualize o banco físico.

Regras obrigatórias

Não recrie as tabelas.

Escolha a operação adequada para cada alteração.

Depois confira as estruturas modificadas.

Exercício Prático 18 – Estrutura Temporária

Contexto Empresarial

Durante a migração dos equipamentos, a equipe criou uma tabela temporária chamada equipamento_importacao.

Ela deverá possuir:

- código;
- descrição;
- quantidade.

Tarefa

Crie a estrutura.

Depois considere duas situações independentes.

Situação A: os dados importados estão incorretos e precisam ser removidos, mas a estrutura será utilizada novamente.

Situação B: depois da conclusão definitiva da migração, a estrutura não terá mais utilidade.

Determine qual operação é adequada para cada situação.

Exercício Prático 19 – Encerramento dos Ambientes de Treinamento

Contexto Empresarial

A etapa de treinamento foi concluída.

Antes de iniciar o projeto oficial, os ambientes utilizados exclusivamente para exercícios deverão ser removidos.

Tarefa

Verifique os bancos existentes no servidor.

Identifique entre eles os bancos utilizados nesta sequência apenas para treinamento e remova-os.

Regras de negócio

dbHorizonte ainda não existe e não deverá ser criado neste exercício.

Os scripts de exclusão deverão considerar a possibilidade de algum ambiente já ter sido removido.

Ao terminar, confira novamente o servidor.

ETAPA 4 — PROJETO INTEGRADOR

Exercício Prático 20 – Retomando o Projeto Antes do SQL

Contexto Empresarial

A **Horizonte Comercial** comercializa produtos para clientes de diversas cidades.

A empresa possui funcionários, fornecedores, produtos organizados por categorias, estoque, pedidos, pagamentos e entregas.

Antes de escrever qualquer comando SQL, a equipe deverá lembrar que o banco físico é consequência do projeto.

Tarefa

A partir das informações fornecidas pelo projeto da Horizonte Comercial:

1. identifique as principais entidades;
2. identifique os relacionamentos existentes;
3. identifique situações 1:N;
4. identifique situações N:N;
5. identifique quais relações associativas deverão aparecer no modelo lógico;
6. confira se o modelo está normalizado antes da implementação.

Regra obrigatória

Não escreva SQL ainda.

O objetivo deste exercício é reforçar que a implementação física somente deverá começar depois que o projeto estiver compreendido.

Exercício Prático 21 – Do Modelo Conceitual para o Lógico

Regras de negócio da Horizonte Comercial

A equipe levantou as seguintes necessidades:

- um estado possui várias cidades;
- cada cidade pertence a um estado;
- clientes podem possuir endereços;
- funcionários ocupam cargos;
- produtos pertencem a categorias;
- fornecedores podem fornecer vários produtos;
- um produto poderá possuir vários fornecedores;
- cada produto possuirá controle de estoque;
- clientes realizam pedidos;
- cada pedido possui um funcionário responsável;
- um pedido possui vários produtos;
- um produto poderá aparecer em vários pedidos;
- pedidos poderão possuir pagamentos;
- cada pagamento utilizará uma forma de pagamento;
- pedidos poderão gerar entregas.

Tarefa

Antes da implementação física, analise essas regras juntamente com o modelo conceitual elaborado para o projeto.

Confira se o modelo lógico representa corretamente os relacionamentos e se as relações necessárias foram obtidas após a normalização.

Atenção

Ainda não é o momento de criar tabelas.

Primeiro devemos saber **o que será implementado e por quê**.

Exercício Prático 22 – Nascimento do Banco Oficial

Contexto Empresarial

O projeto foi aprovado.

O modelo lógico está definido e a equipe recebeu autorização para iniciar a implementação física.

Tarefa

Crie o banco oficial:

dbHorizonte

Confirme sua criação.

Deixe-o selecionado para o início da implementação.

Regras obrigatórias

Inclua um comentário de múltiplas linhas identificando o início da implementação física.

Esse banco será utilizado durante todo o restante da disciplina e **não deverá ser excluído**.

Exercício Prático 23 – Implementando Localização

Modelo lógico aprovado

ESTADO

- id_estado — identificador;
- sigla — exatamente 2 caracteres, obrigatória e única;
- nome — até 50 caracteres, obrigatório.

CIDADE

- id_cidade — identificador;
- nome — até 100 caracteres, obrigatório;
- id_estado — referência obrigatória a ESTADO.

Tarefa

Transforme essa parte do modelo lógico em estruturas físicas.
Depois confira as tabelas e suas estruturas.

Exercício Prático 24 – Implementando Clientes e Endereços

CLIENTE

- código interno gerado automaticamente;
- nome ou razão social — até 150 caracteres, obrigatório;
- documento — até 14 caracteres, obrigatório e único;
- tipo de pessoa — F ou J;
- telefone — opcional;
- e-mail — opcional;
- data de nascimento — opcional;
- data e hora do cadastro;
- situação inicialmente ativa.

ENDEREÇO

- identificador;
- cliente;
- logradouro;
- número;
- complemento;
- CEP;
- cidade.

Regras de negócio

Todo endereço pertence a um cliente existente.

Todo endereço pertence a uma cidade existente.

Logradouro, número e CEP são obrigatórios.

Complemento é opcional.

O número poderá conter valores como 120, 120A ou S/N.

Tarefa

Implemente essa parte do projeto.

Exercício Prático 25 – Estrutura Organizacional

Modelo lógico

CARGO

- identificador;
- nome obrigatório e único;
- salário-base não negativo.

FUNCIONARIO

- identificador gerado automaticamente;
- nome obrigatório;
- CPF obrigatório e único;
- data de admissão obrigatória;
- horário de entrada;
- cargo obrigatório;
- situação inicialmente ativa.

Regra de negócio

Um cargo poderá ser ocupado por vários funcionários.
Cada funcionário deverá possuir um cargo existente.

Tarefa

Implemente o modelo.

Depois confira se o relacionamento definido no projeto aparece corretamente no banco físico.

Exercício Prático 26 – Catálogo Comercial

Modelo lógico

CATEGORIA

- identificador;
- nome obrigatório e único;
- descrição opcional.

PRODUTO

- identificador gerado automaticamente;
- nome obrigatório;
- categoria obrigatória;
- preço de venda com duas casas decimais;
- ano de fabricação;
- estoque mínimo;
- situação inicialmente ativa.

Regras de negócio

O preço não poderá ser negativo.

O estoque mínimo não poderá ser negativo.

Todo produto deverá pertencer a uma categoria existente.

Tarefa

Implemente as relações.

Exercício Prático 27 – Fornecedores e Produtos

Contexto do modelo conceitual

FORNECEDOR N : N PRODUTO

Um fornecedor poderá fornecer vários produtos.

Um produto poderá ser adquirido de diferentes fornecedores.

Após a transformação e normalização, surgiu a relação associativa:

PRODUTO_FORNECEDOR

Ela deverá registrar:

- produto;
- fornecedor;
- código utilizado pelo fornecedor para o produto;
- preço de custo.

FORNECEDOR

Também deverá possuir:

- identificador;
- razão social;
- CNPJ obrigatório e único;
- telefone opcional;
- e-mail opcional;
- situação inicialmente ativa.

Regras de negócio

Produto e fornecedor deverão existir.

O mesmo produto não poderá ser relacionado duas vezes ao mesmo fornecedor.

O preço de custo não poderá ser negativo.

Tarefa

Implemente essa parte do projeto.

Exercício Prático 28 – Controle de Estoque

Projeto

Cada produto deverá possuir um único controle de estoque.

O controle deverá armazenar:

- produto;
- quantidade atual;
- data e hora da última atualização.

Regras de negócio

Não poderá existir estoque de produto inexistente.

A quantidade não poderá ser negativa.

Um produto não deverá possuir dois controles diferentes de estoque.

Tarefa

Defina e implemente a estrutura adequada.

Atenção

O projeto apresenta a regra.

Cabe a você decidir como representar sua identificação e suas restrições no banco físico.

Exercício Prático 29 – Pedidos

PEDIDO

- identificador;
- cliente;
- funcionário responsável;
- data e hora;
- percentual de desconto;
- situação.

Regras de negócio

Cliente e funcionário deverão existir.

O desconto deverá permanecer entre 0 e 100.

Quando não houver desconto informado, considere zero.

Tarefa

Implemente a relação.

Exercício Prático 30 – Itens do Pedido

Contexto do projeto

O modelo conceitual possui:

PEDIDO N : N PRODUTO

Durante a transformação para o modelo lógico surgiu:

ITEM_PEDIDO

- pedido;
- produto;
- quantidade;
- valor unitário.

Regras de negócio

Pedido e produto deverão existir.

Quantidade deverá ser maior que zero.

Valor unitário não poderá ser negativo.

O mesmo produto não poderá aparecer duas vezes no mesmo pedido.

Tarefa

Implemente a relação associativa respeitando as regras definidas no projeto.

Exercício Prático 31 – Pagamentos

Projeto lógico

FORMA_PAGAMENTO

- identificador;
- descrição obrigatória e única;
- situação inicialmente ativa.

PAGAMENTO

- identificador;
- pedido;
- forma de pagamento;
- valor pago;
- data e hora;
- situação.

Regras de negócio

Pedido e forma de pagamento deverão existir.

O valor pago deverá ser maior que zero.

Tarefa

Implemente essa parte do projeto.

Exercício Prático 32 – Entregas

Projeto lógico — ENTREGA

- identificador;
- pedido;
- endereço;
- data prevista;
- data de envio;
- data efetiva da entrega;
- situação.

Regras de negócio

Pedido e endereço deverão existir.

A data prevista é obrigatória.

Data de envio e data efetiva poderão inicialmente não possuir valor.

Tarefa

Implemente a estrutura.

Exercício Prático 33 – Auditoria Projeto x Banco Físico

Contexto Empresarial

A primeira versão estrutural do sistema Horizonte foi concluída.

Antes de prosseguir, a equipe realizará uma auditoria.

Tarefa

Utilizando SQL, confira:

- o banco utilizado;
- todas as tabelas existentes;
- a estrutura de cliente;
- a estrutura de produto;
- a estrutura de pedido;
- a estrutura de item_pedido;
- a estrutura de pagamento;
- a estrutura de entrega.

Depois compare essas estruturas com o modelo lógico aprovado.

O banco físico representa o projeto?

Se alguma regra definida no projeto não estiver representada, faça a correção necessária.

Exercício Prático 34 – O Negócio Mudou

Contexto Empresarial

Depois da apresentação da primeira versão do sistema, a Horizonte Comercial solicitou algumas alterações. As mudanças representam **novas regras de negócio** e deverão ser incorporadas ao banco físico já existente.

Novas regras

- O nome do cliente deverá aceitar até **200 caracteres**.
- O cliente passará a possuir `observacao`, com até **250 caracteres** e preenchimento opcional.
- O telefone do fornecedor deixou de ser necessário.
- Em `produto`, `ano_fabricacao` passará a se chamar `ano`.

Tarefa

Atualize o banco físico para que ele continue representando corretamente o projeto e as novas regras de negócio.

Ao terminar, confira as estruturas modificadas.

Regras obrigatórias

- Não exclua as tabelas para criá-las novamente.
- Preserve as demais informações definidas no projeto.
- Analise cada mudança e escolha a operação SQL adequada.

Exercício Prático 35 – Mudanças em Produtos

Novas regras de negócio

Produto deverá possuir:

- peso, aceitando casas decimais;
- marca, com até 80 caracteres.

Inicialmente o peso será opcional.

A marca será obrigatória.

Tarefa

Atualize a estrutura.

Depois confira o resultado.

Exercício Prático 36 – Outra Mudança de Regra

A empresa decidiu que o peso do produto não poderá mais ficar sem preenchimento.

Tarefa

Adapte a estrutura existente.

Regra obrigatória

Não crie outra coluna.

Modifique aquela que já representa essa informação.

Exercício Prático 37 – Informação Descontinuada

A empresa decidiu deixar de controlar o ano de fabricação dos produtos.

Tarefa

Atualize o banco físico para refletir a nova versão do projeto.

Depois confira a estrutura.

Exercício Prático 38 – Estrutura Temporária de Importação

Durante uma migração, será necessária uma estrutura chamada:

`produto_importacao`

Ela armazenará temporariamente:

- código;
- descrição;
- quantidade.

Tarefa

Crie a estrutura.

Depois considere:

Situação A: os dados precisam ser apagados, mas a estrutura será reutilizada.

Situação B: posteriormente a estrutura inteira deixará de ser necessária.

Determine a operação adequada para cada situação.

Exercício Prático 39 – Alteração de Terminologia

Contexto Empresarial

A equipe responsável pela logística da Horizonte Comercial realizou uma revisão dos termos utilizados no projeto.

A partir de agora, a estrutura denominada `entrega` deverá passar a se chamar:

`expedicao`

A mudança é apenas de terminologia. A estrutura e sua finalidade permanecem as mesmas.

Tarefa

Atualize o banco físico para representar a nova terminologia adotada no projeto.

Depois, confira as tabelas existentes e verifique se a alteração foi realizada.

Regras obrigatórias

- Não exclua a tabela para criá-la novamente.
- Os dados e a estrutura já definidos deverão ser preservados.
- Escolha a operação SQL adequada para realizar a mudança.

Exercício Prático 40 – Desafio Final de DDL

Contexto Empresarial

Um analista recebeu o projeto Horizonte e precisa verificar se o ambiente físico está consistente com sua versão atual.

Ele não possui o histórico dos comandos executados.

Possui apenas:

- as regras de negócio;
- o modelo lógico atualizado;
- acesso ao banco físico.

Tarefa

Faça uma auditoria completa da estrutura utilizando os comandos SQL estudados.

Confira bancos, tabelas e estruturas.

Compare:

Projeto aprovado

↓

Modelo lógico

↓

Banco físico

Caso encontre divergências, faça as alterações necessárias sem destruir estruturas corretas.

Regra principal

Não escolha um comando porque “este é o exercício de determinado comando”.

Primeiro identifique **o problema que precisa ser resolvido**.

Depois escolha o recurso SQL adequado.

Futuro....

INDEX – Índices

Agora chegamos a um recurso um pouco diferente.

Imagine um livro com centenas de páginas.

Se você quiser encontrar um determinado assunto, poderá procurar página por página.

Mas existe uma maneira mais rápida:

consultar o índice do livro.

Nos bancos de dados, a ideia de um índice também está relacionada a **facilitar a localização das informações**.

Um **índice** é uma estrutura utilizada pelo SGBD para ajudar a localizar registros com maior eficiência em determinadas consultas.

No MySQL, índices podem permitir que o servidor encontre rapidamente as linhas correspondentes a determinados valores, evitando a necessidade de percorrer todas as linhas em algumas situações.

CREATE INDEX

Podemos criar um índice utilizando:

```
CREATE INDEX
```

Sintaxe

```
CREATE INDEX <nome_indice>  
ON <nome_tabela> (<nome_coluna>);
```

Vamos entender:

- **CREATE** → criar.
- **INDEX** → objeto que desejamos criar.
- **<nome_indice>** → nome que daremos ao índice.
- **ON** → indica em qual tabela o índice será criado.
- **<nome_tabela>** → tabela.
- **<nome_coluna>** → coluna que participará do índice.

Exemplo

Imagine: tbAluno

com uma coluna: Nome

Podemos criar:

```
CREATE INDEX idx_Nome  
ON tbAluno (Nome);
```

Neste exemplo:

- **idx_Nome** → nome do índice.
- **tbAluno** → tabela.
- **Nome** → coluna indexada.

Para que serve o índice?

Imagine uma tabela muito grande:

tbAluno

```
1      Carlos  
2      Maria  
3      João  
...
```

Sem entrar ainda em detalhes de otimização, podemos entender que um índice pode ajudar o MySQL a localizar valores de uma coluna de maneira mais eficiente.

A documentação oficial destaca que índices são uma das principais formas de melhorar o desempenho de operações `SELECT`, especialmente quando as colunas indexadas participam das condições da consulta.

Mas existe um detalhe:

Não significa que devemos criar índices em todas as colunas.

Os índices também ocupam espaço e precisam ser atualizados quando os dados são inseridos, alterados ou excluídos. Portanto, índices desnecessários também possuem custo.

Um detalhe interessante

Quando criamos:

`PRIMARY KEY`

ou:

`UNIQUE`

o MySQL também utiliza estruturas de índice para essas chaves. A documentação do MySQL inclui `PRIMARY KEY`, `UNIQUE` e `INDEX` entre seus tipos comuns de índices.